闽江学院

本科毕业论文(设计)

题　　目	基于 SpringBoot 的闽江学院校园论坛的设计与实现
学 生 姓 名	谢荣臻
学　　号	3182308205
学　　院	数学与数据科学学院（软件学院）
年　　级	2018
专　　业	数据科学与大数据技术
指 导 教 师	林苗
职　　称	讲师
完 成 日 期	年　月　日

闽江学院毕业论文（设计）诚信声明书

本人郑重声明:

兹提交的毕业论文（设计）《基于 SpringBoot 的闽江学院校园论坛的设计与实现》，是本人在指导老师 林苗 的指导下独立研究、撰写的成果；论文（设计）未剽窃、抄袭他人的学术观点、思想和成果，未篡改研究数据，论文（设计）中所引用的文字、研究成果均已在论文（设计）中以明确的方式标明；在毕业论文（设计）工作过程中，本人恪守学术规范，遵守学校有关规定，依法享有和承担由此论文（设计）产生的权利和责任。

声明人（签名）：

年　月　日

摘　　要

随着信息技术的迅猛发展，网络信息交流已融入到人们的生活中。校园论坛是以学生为主体，以学术交流与信息共享为根本的一个网络社区，它不仅可以为在校师生的交流互动提供便利，还可以为广大校友提供沟通和交流的渠道，提高学生的校园归属感。校园论坛不仅是学校信息化程度的标识，也在一定程度上代表了学校的软实力，因此，建立一个具有本校特色的校园论坛是很有必要的。本系统以闽江学院校园论坛的设计和实现为目标，使用 Java 语言，采用 SpringBoot 技术，基于 MVC 架构模式进行设计开发。该系统界面简洁易操作，具有较好的安全性、较高的并发性和较强的可扩展性。

关键词：SpringBoot；校园论坛；MVC

Abstract

With the rapid development of information technology, network information exchange has been integrated into people's life. Campus forum is an online community with students as the main body and academic exchange and information sharing as the root. It can not only facilitate the exchange and interaction of students and teachers, but also provide communication and exchange channels for alumni and improve students' sense of belonging to the campus. The campus forum is not only a sign of the degree of informationization of the school, but also represents the soft power of the school to a certain extent, therefore, it is necessary to establish a campus forum with the characteristics of our school. The system is designed and implemented in Java language, using SpringBoot technology, and based on the MVC architecture model. The system is simple and easy to operate, with good security, high concurrency and strong scalability.

Key words：SpringBoot；Campus Forum；MVC

目　　录

1　引言
1.1 研究背景及意义 - 1 -
1.2 研究现状 - 1 -
1.3 本文主要工作 - 2 -
2　系统开发语言与技术介绍
2.1 Java 语言介绍 - 3 -
2.2 开发环境介绍 - 3 -
2.3 开发容器 docker 介绍 - 3 -
2.4 开发框架介绍 - 3 -
2.5 开发工具 Idea 简介 - 5 -
2.6 关系型数据库 MySQL 简介 - 5 -
2.7 非关系型数据库 Redis 简介 - 5 -
3　系统分析
3.1 系统需求分析 - 6 -
3.1.1 功能需求 - 6 -
3.1.2 性能需求 - 6 -
3.2 可行性分析 - 7 -
3.3 系统模块分析 - 8 -
4　数据库设计
4.1 数据库概念结构设计 - 10 -
4.2 数据库逻辑结构设计 - 13 -
4.3 数据库连接与实现 - 16 -
5　系统的详细设计与实现
5.1 系统软件架构 - 17 -
5.1.1 注册模块和登录模块 - 17 -
5.1.2 个人信息模块 - 19 -
5.1.3 版块管理 - 21 -
5.1.4 帖子管理 - 22 -
5.1.5 帖子收藏、回复和浏览以及排行榜 - 24 -
6　系统部署与测试
6.1 系统测试 - 27 -
6.1.1 系统测试目的 - 27 -
6.1.2 系统测试过程与结果 - 27 -

6.2 系统部署 - 29 -
6.3 部署过程遇到的问题和解决方法 - 30 -
6.4 系统存在的不足 - 30 -
7 结语
参考文献 - 33 -
致谢 - 34 -
附录 - 35 -

基于 SpringBoot 的闽江学院校园论坛的设计与实现

谢荣臻

（闽江学院 数学与数据科学学院（软件学院），福建 福州 350108）

1 引言

1.1 研究背景及意义

随着 Internet 的飞速发展，在二十一世纪的今天，互联网已成为人们快速获取、发布和传递信息的重要渠道，它在人们政治、经济、生活、学习等各个方面发挥着重要的作用。目前，国内外大部分机关、学校、公司、企业都已经或正在建立自己的论坛。校园论坛是以学生为主体，以学术交流与信息共享为根本的一个网络社区，它不仅可以为在校师生的交流互动提供便利，还可以为广大校友提供沟通和交流的渠道，因此，它已成为学校信息化建设的重要组成部分。

另一方面，在信息爆炸的今天，搜集数据、分类信息显得尤其重要，为了满足学生群体的学习、生活需要，扩大学生的交际范围，丰富学生的校园生活，实现信息与资源的交流与共享，开发一个操作便捷、功能强大的校园论坛系统是很有必要的。借助该系统，学生用户可以及时获取校园新闻、热点资讯、教学资源等信息，同时可以刊登“交友”、”二手转让”、“学习互助”、“失物招领”等信息，为学生的生活和学习带来了极大的便利，同时也为他们搭建了一个围绕校园热点、碰撞思想火花的交流平台。

1.2 研究现状

目前闽江学院学生获取校园信息的渠道主要包括微信公众号、微博、朋友圈等，获取信息的渠道过多，且获取的内容过于凌乱且不全面。现在较为热门的信息社区如百度贴吧、空间等，也都是规模相当大的社区网络平台，不过它们所涵盖的信息范畴都太大了，并不能适应单纯学生间的沟通需求，同时它们都缺少了校园元素，所以学生们使用起来也不方便。具有校园元素的学校官网，是学校官方的网站，一般只刊登学校的官方、重大的事件，与学生的校园生活关系不紧密。

高校校园论坛是基于高校校园网络，是面向高校大学生并由学生自主管理的

网络系统。近年来，随着高校网络的迅猛发展，高校校园论坛已经成为当代大学生交流思想、学习、生活的重要平台，同时也成为校园文化不可或缺的重要组成部分，对构建和谐校园文化具有重要的作用。校园论坛的形式与内容多种多样，开发的工具也可以根据开发者的喜好选择。本文基于 SpringBoot 技术的闽江学院校园校园论坛系统，实现了校园论坛论坛的基本功能，主要是针对校园的学生，本系统的内容与校园学生的需求更加吻合。

1.3 本文主要工作

本文主要分析了校园论坛系统的需求分析，通过调研其他高校校园论坛以及收集学生群体意见，得到用户精确的需求和准备的业务流程，最后根据需求分析进行系统的设计与研发思路。

第 1 章是引言，介绍了校园论坛系统的课题背景及意义、校园论坛的研究现状，以及本论文研究的主要内容。

第 2 章是开发语言与技术简介，介绍了开发系统的编程语言和相关技术的简介，阐述 SpringBoot 框架、Docker 容器技术、Thymeleaf 模板引擎、MySQL 数据库技术、Redis 非关系型数据库技术、B/S 架构等。

第 3 章是系统分析，提出了校园论坛系统的系统需求分析、可行性分析和系统功能模块分析。

第 4 章是数据库设计，主要是数据库表之间关系图和数据库表结构的具体实现，以及表之间的关系。

第 5 章是系统的详细设计与实现，系统模块的实现与调试，描述了各业务模块的功能设计，开发了各业务模块，并给出了各业务模块的截图和关键代码。

第 6 章是系统测试与部署，提出了完成的校园论坛系统的测试目的和功能测试，同时将系统部署至线上环境，并针对可能出现问题进行系统架构优化。

最后对全文进行了总结，并对本文的研究结果进行了展望。

2 系统开发语言与技术介绍

2.1 Java 语言介绍

Java 是一种基于类、面向对象的编程语言，旨在尽可能减少实现依赖项。一种为开发人员编写的通用编程语言，只要编写一次就可以在任何地方运行，编译后的 Java 代码可以在所有支持 Java 的平台上运行。Java 应用程序被编译成可以在任何 Java 虚拟机（Java Virtual Machine, JVM）上运行的字节码。Java 的语法类似于 c/c++。但摒弃了 c/c++中晦涩难懂且危险的指针操作，由 JVM 中的垃圾回收器进行未引用对象的处理，垃圾收集器可以重新收集未引用的对象。因此，Java 通过处理内存管理使程序员的生活变得轻松。同时，Java 社区成熟强大，并且 Java 版本仍在持续更新中[1]。

随着 Java 版本的不断迭代，Java1.8 版本由于其稳定的特性深受企业的欢迎，同时 Java1.8 在 Java1.7 的基础上加入了 Lambda 表达式、Stream 流等新特性，可以帮助减少冗余的代码。尽可以敲更少、更整洁的代码、更高效的完成业务。因此本系统也是基础 Java1.8 进行开发[2]。

1.3 开发环境介绍

目前主流的服务器有 Linux server 和 Windows server，与 Windows server 相比，因为 Linux Server 的开源性[3]、可移植性[4]、稳定性[5]、安全性[6]、灵活性[7]以及良好的社区支持[8]，使得百分之八十的企业和个人选择 Linux 服务器作为开发环境，因此本系统是基于阿里云服务器，centos7 开发环境进行开发。

1.4 开发容器 docker 介绍

Docker 是一款基于 Go 语言的开发、交付和运行应用程序的开放平台。Docker 能够将应用程序与基础架构分离，以实现快速软件交付[9]。在 Docker，可以像管理管理应用程序一样管理基础架构。通过利用 Docker 的方法快速交付、测试和部署代码，可以显著减少在生产环境中编写代码和运行代码之间的延迟[10]。

2.4 开发框架介绍

1. SpringBoot 框架

SpringBoot 是目前比较流行的企业应用开发框架，它在传统的 SSM（Spring、SpringMVC、Mybatis）的基础上，简化了配置文件的编写，将原有的大量 xml 配置文件简化为 Java 类配置，同时 SpringBoot 官方为 SpringBoot 框架集成了大量常用的组件 jar 包，并根据不同的 SpringBoot 版本定制化的选择版本，避免兼容性问题。开发人员只需要关注业务层面的开发即可[11]。

2. Mybatis 框架

MyBatis 是一个优秀的持久层框架，它封装了 jdbc 操作数据库的过程，因此开发人员只需要关注 SQL 本身，而不需要花费精力来处理 jdbc 复杂的过程代码，

Mybatis 通过 xml 或注释配置要执行的各种 statement，并通过 java 对象和 statement 中的 sql 之间的映射生成最终执行的 sql 语句。最后，mybatis 框架执行 sql 并将结果映射到 java 对象并返回[13]。图 2-1 是 Mybatis 执行流程图。

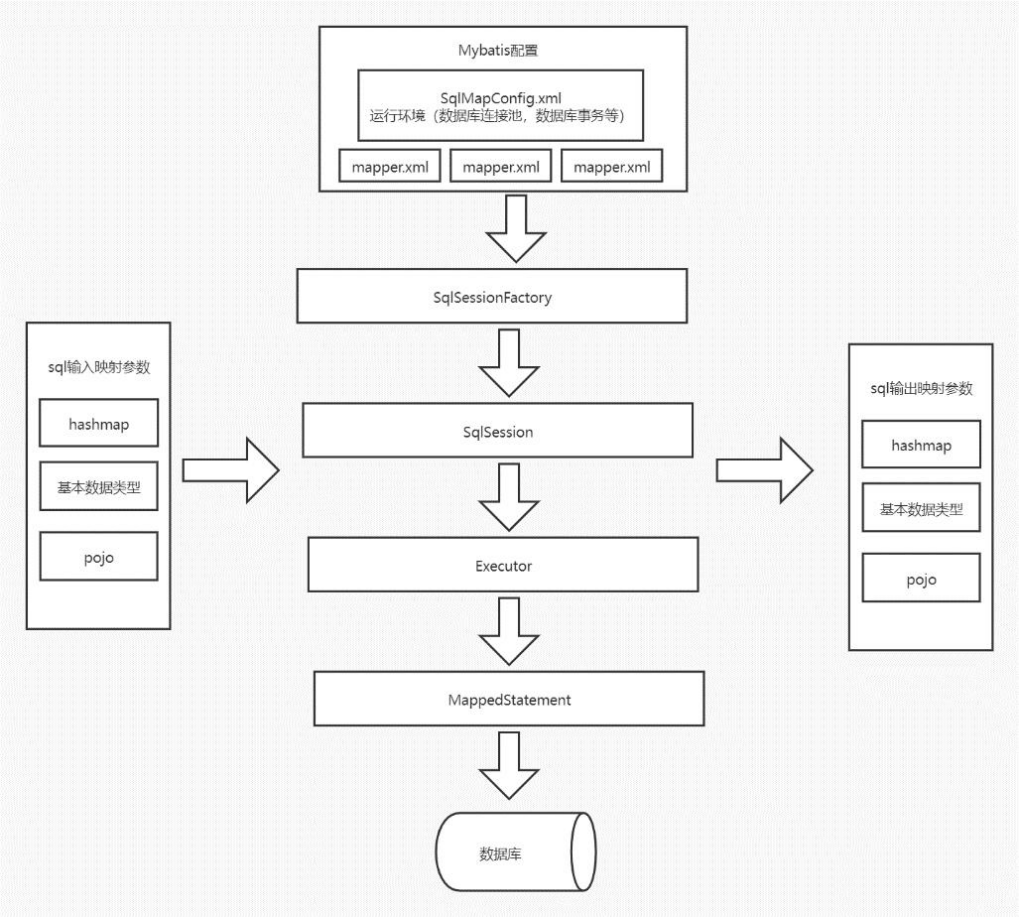

图 0-1 Mybatis 执行流程图

2.5 开发工具 Idea 简介

IntelliJ IDEA 具有一些生产效率最高的 Java 代码完成功能。即使他不知道特定类，成员或任何其他资源的确切名称，它的预测算法也可以准确地假定编码人员要尝试键入的内容，并为他完成编码。

IDEA 内置了大量开发人员工具。IDEA 为开发人员提供了一个了不起的工具集，其中包括反编译、Docker 支持、字节码查看器、版本控制（Git、SVN 等）、FTP、构建工具（Maven、Gradle）、数据库工具等许多其它工具。

IDEA 符合开发人员人体工程学，其以编辑者为中心的环境、万能的快捷方式、内联的调试器等，深受 Java 开发人员喜爱，大幅提高 Java 程序开发的效率[14]。

2.6 关系型数据库 MySQL 简介

MySQL 是一个开源的 SQL 数据库服务器。所谓的 SQL 即结构化查询语言，在很大程度上极大地方便了数据的存储、更新和获取，使用非常方便，并为人们在获取信息时节省了很多不必要的麻烦，以各种方式连接数据库。MySQL 可以根据用户的不同需求选择不同的存储引擎。同时，它还具有很强的平台独立性，可以在多种操作系统下运行。MySQL 作为一个关系数据库，由于其开源和跨平台的特点，已经越来越多地被广大用户所使用[15]。

MySQL 数据库是开源的，可以在官方网站上免费下载和安装。考虑到各种因素，选择使用 MySQL 数据库来开发这个系统。

2.7 非关系型数据库 Redis 简介

Redis 是开源的、基于键值对存储的具有高性能的非关系型数据库。它具有极高的性能，支持每秒 110000 次的读操作以及 81000 次的写操作；它提供丰富的数据类型，同时提供原子性操作及事务，使得 redis 能够灵活的运用于各种业务场景。

在最新版 redis 中，加入了服务器主从热备和分片操作，以确保系统稳定性和能够应对海量数据和高并发，从而实现分布式支持。

因此, Redis 常被用于缓存、消息队列等。正因为有了 Redis 的存在, 使得系统能够从单节点的项目拓展成为一个多节点的集群项目, 也使得项目可以经受起高并发的挑战, 同时减轻项目服务器和数据库服务器的压力。

3 系统分析

3.1 系统需求分析

3.1.1 功能需求

校园论坛系统的功能需求主要包括以下几个方面:它可以提供校园论坛的注册新用户、登录、浏览主题、发布回复主题、修改个人资料、删除帖子等功能。下面将使用用例说明。

3.1.2 性能需求

该系统性能需求如下：

1.安全性需求

为了保证用户在登录过程中的任何一个环节的安全性,引入 MD5 加密算法，当用户输入相应数据后，前端通过 MD5 算法对隐私数据进行加密，通过 HTTP 请求将数据发送给后端，此时的数据包中的数据是经过加密后的，能够避免抓包获取用户隐私数据等不正当行为，避免个人信息的泄露等安全问题的发生。

2.时间性需求

在请求访问过程中，要尽可能的控制响应时间，在一定程度上对网络 IO 的延时时间 TTL 以及数据库读写时间进行监控，实现浏览页面时间延迟小于 1 秒，更新数据文件时间响应时间小于 100 毫秒

3.可靠性需求

给用户提供密码保护功能以保证在忘记密码的时候给予帮助。用户忘记密码时，用户可以通过提供相应的数据信息，与数据库中进行比对，若比对成功，将由系统重置数据库中密码。

4.系统容量需求

数据库可存储 5 年的用户信息以及 3 个月的论坛文章(发帖，回复等)，允许 5000 个终端同时工作，并且需要考虑多用户高并发请求情况。

5.可移植可扩展性需求

为了使系统能够在不同平台兼容移植，引入 docker 技术，只需要在不同平台安装 docker，由 docker 构建镜像就能够在不同的平台进行移植，同时为了使在后续的开发过程中，便于对系统进行功能的扩充升级，在编写代码时要充分考

虑代码的耦合度，尽可能的降低耦合，实现可扩展性。

6. 界面友好需求

对于一个系统，应降低用户的上手学习难度，因此在设计时，就应当充分考虑界面友好型，降低用户使用的学习时间成本。

3.2 可行性分析

本文以 Java 为核心，Idea 为开发工具，SpringBoot 整合 Mybatis 与 MySQL 数据库建立数据连接关系；使用 thyleaf 前端引擎魔板以及 JavaScript + jQuery 等技术实现效果展示以及数据渲染；采用 MVC 架构模式[18]，对模型、视图、控制器进行分离（控制器用于请求转发，视图层仅用于数据展示，模型用于数据库操作），并且对业务层、数据持久层、控制器进行分离（控制器用于请求转发，业务层专注于业务实现，数据持久层仅需要关注数据库层面），一定程度上实现解耦，每个模块相互独立，有利于项目的分工以及后期项目的迭代和扩展，同时降低了维护的成本及时间。图 3-1 为 MVC 架构模式图，图 3-2 为 MVC 架构模式在项目中实现流程图。

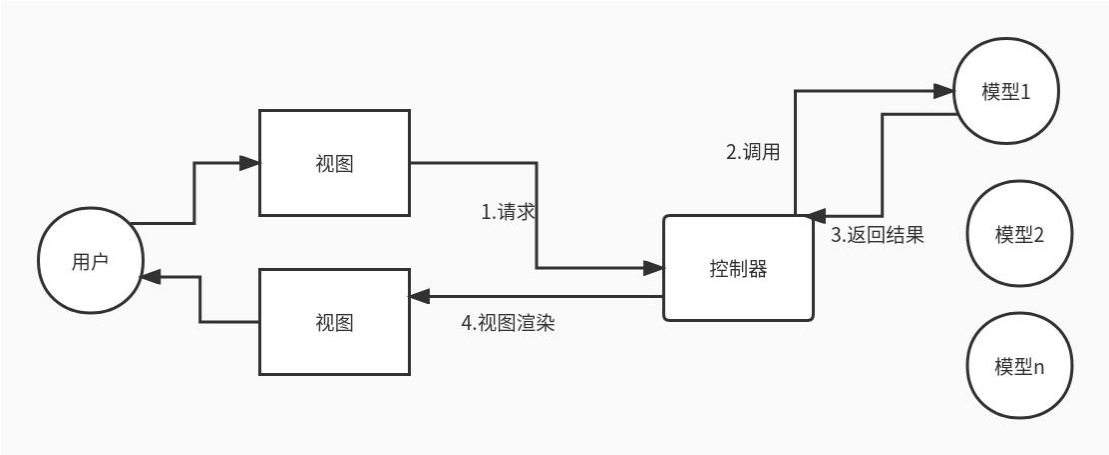

图 3-1 MVC 架构模式图

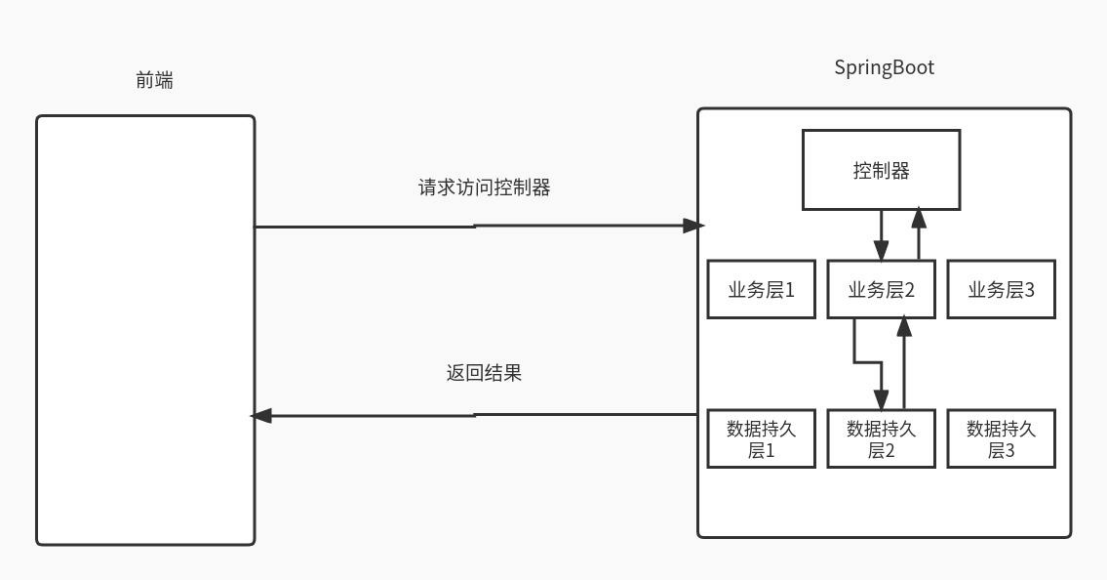

图 3-2 MVC 架构模式在项目中实现流程图

3.3 系统模块分析

随着互联网的发展，学生获取校园信息的渠道越来越多，但获取的内容过于凌乱且不全面，学生无法及时且完整的获得自己所需要的信息。因此，为了解决信息散乱的问题，充分整合校园信息，开发一款校园论坛非常有必要，让师生可以在校园论坛上及时发布相关的校园信息，充分整合信息资源，让师生可以减少获取、整合信息的时间成本。

本系统意在为在校师生以及校友提供一个信息发布和获取，同时能够进行在线交流的网站。因此该系统分为三大模块：

（1）用户模块：用户模块主要包含用户的相关信息以及用户操作日志，让信息获取人能够及时、有效的联系到信息发布者，在一定程度上也对恶意发表言论者进行溯源，及时发现，及时处理。

（2）内容模块：内容模块主要包含论坛帖子的发布、更新、删除等操作。

（3）交流模块：目前的交流模块仅包含用户在帖子中用户之间的评论、收藏等

用户进入本系统页面后，可以无需进行登录即可观看相关信息，但在发表相关言论等对数据库进行写操作时，系统会跳转到相应的登录界面，只有登录了的用户方法发表言论，以防止某些用户恶意操作。图 3-3 为系统业务流程图。

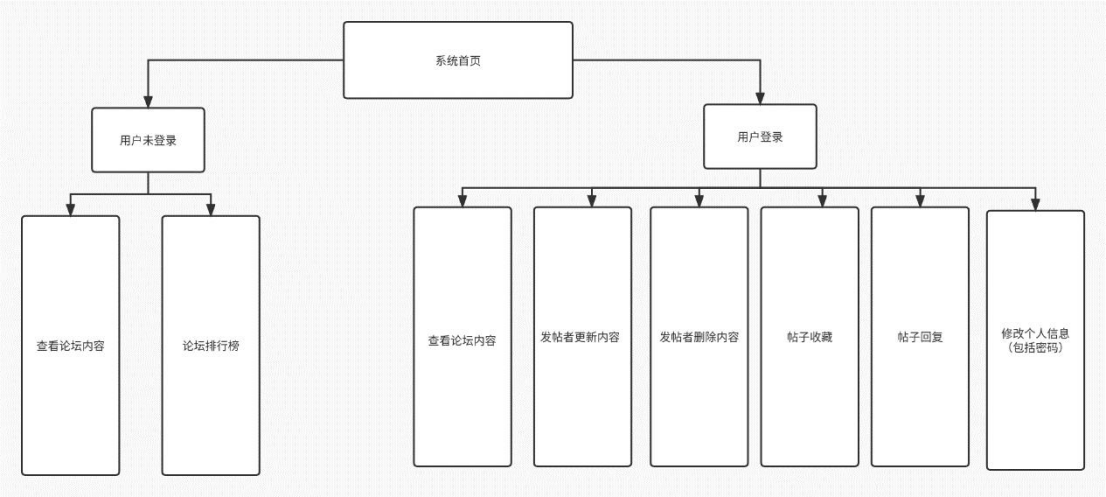
系统首页
用户未登录
用户登录
查看论坛内容
论坛排行榜
查看论坛内容
发帖者更新内容
发帖者删除内容
帖子收藏
帖子回复
修改个人信息
（包括密码）

图 3-3 系统业务流程图

4 数据库设计

4.1 数据库概念结构设计

概念模型是一个以图形方式表示各种实体之间关系的模型，数据库设计首先要完成对实体模型的创建，E－R 图是数据概念模型设计的典型工具。它用矩形框和框中的文本表示数据实体，用椭圆形框表示实体的属性，用菱形框表示实体之间的连接，用线段表示彼此之间的连接关系。根据对系统的分析和数据库数据项的分析，可以设计出各实体之间关系的 E-R 图，通过 E-R 图可以清晰的看出各个实体之间的联系。系统 E-R 图如图 4-1 所示。

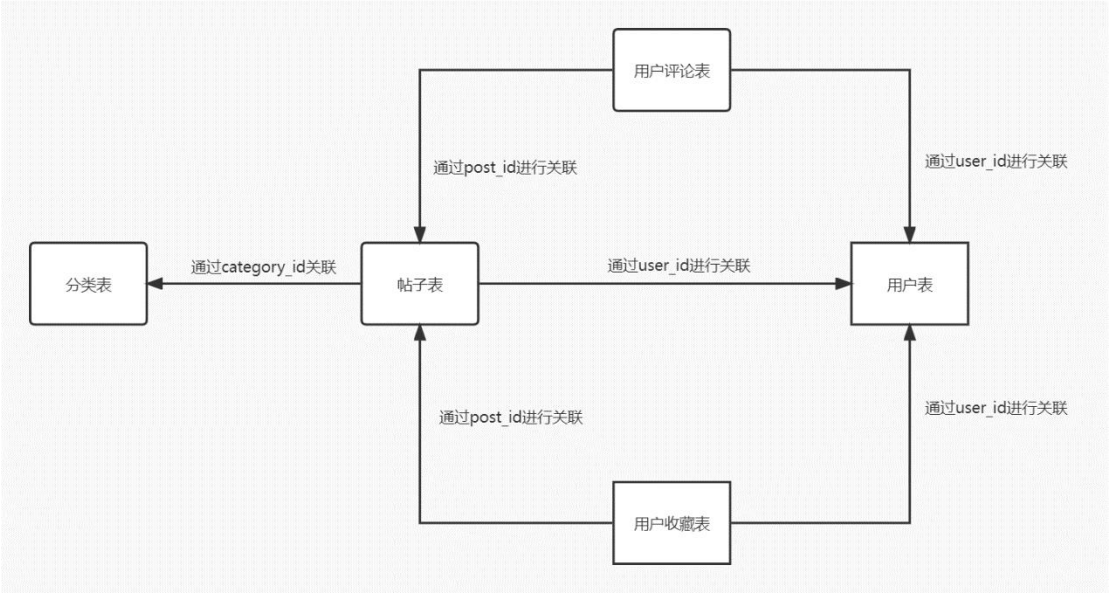

图 4-1 系统 E-R 图

由于外键设置对于系统资源的消耗过大，且对于后期数据库的拓展与维护成本较高，因此，本系统放弃数据库层面的外键约束，通过代码层面的约束对表之间进行关联，以便于后期系统的升级维护和拓展。

本系统对应的实体 E-R 图如下：

1. 相关用户表 E-R 图，在存储用户敏感信息的时候，采取 md5 加密算法对密码进行加密，以便用户的隐私得到充分的保障。同时，在存储有关图片的时候，为减小数据库负担，将图片存入文件系统中，而数据库中只要存储对应文件所在的 url 即可，由前端通过 url 直接访问。相关用户表 E-R 图如图 4-2 所示。

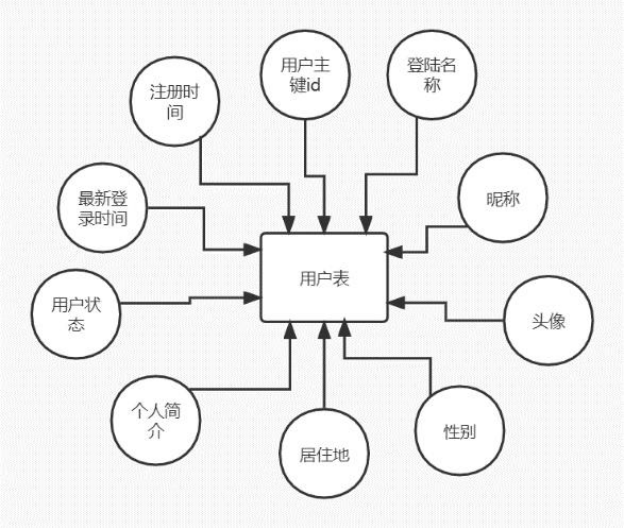

图 4-2 相关用户表 E-R 图

2. 分类目录表 E-R 图描述了分类目录实体的所有属性，其中分类排序值作为逻辑外键与其它相关联表进行链接，并通过是否删除标识进行软删除标记。分类目录表 E-R 图如图 4-3 所示。

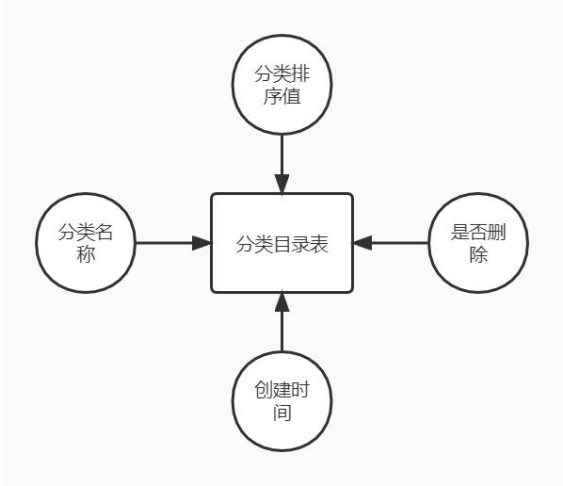

图 4-3 分类目录表 E-R 图

3. 用户收藏表 E-R 图记录了对应用户的收藏记录信息，由于收藏者和帖子之间存在多对多关系，因此采用用户收藏表，由于记录相关收藏者和对应的帖子多对多关系。用户收藏表 E-R 图如图 4-4 所示。

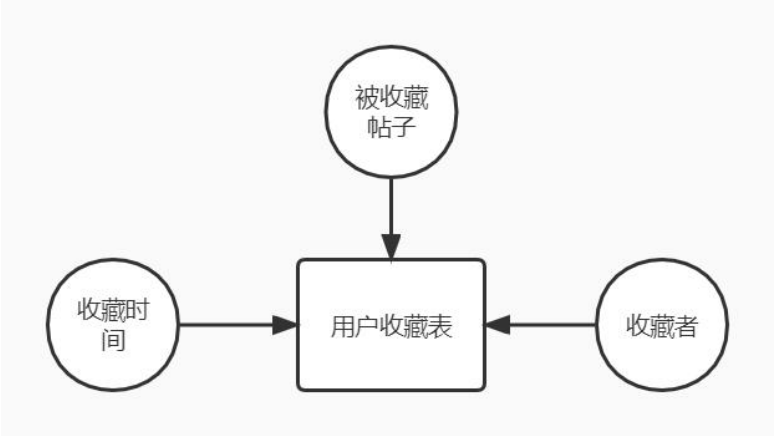

图 4-4 用户收藏表 E-R 图

4. 相关帖子表 E-R 图记录了相关帖子的信息，包括标题、发布者、创建时间、收藏量、评论量等，发布者 id 作为与用户表相关联的逻辑外键，帖子分类 id 作为与分类目录表的逻辑外键。相关帖子表 E-R 图如图 4-5 所示。

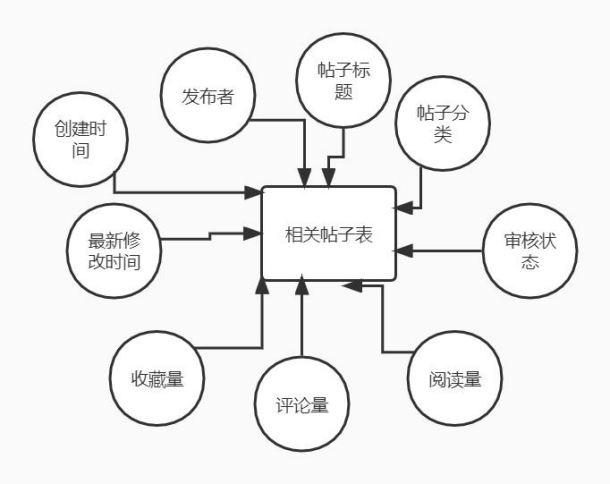

图 4-5　相关帖子表 E-R 图

5. 用户评论表 E-R 图以评论者 id 作为逻辑外键，与用户表进行关联，同时通过树形结构的方式记录上一级评论人，通过递归的方式依次获取所有的评论用户。用户评论表 E-R 图如图 4-6 所示。

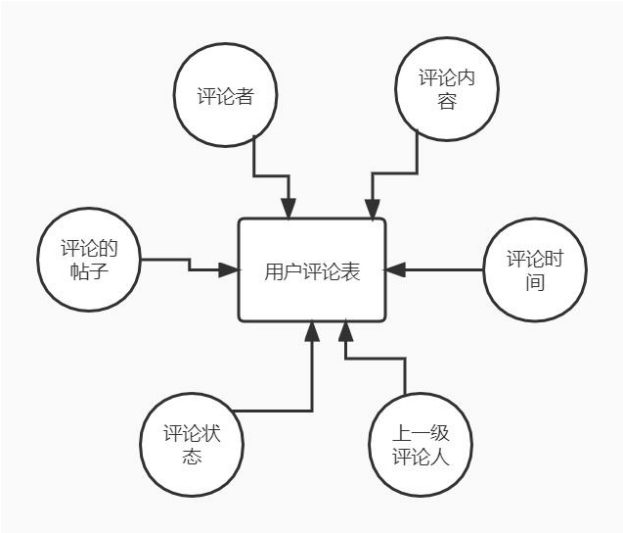

图 4-6 用户评论表 E-R 图

4.2 数据库逻辑结构设计

数据库的逻辑结构就是通过对数据库的概念设计中的实体的描述和分析，对各个实体中的所有属性都做进一步的分析，让各个实体间的关系更加明确，从而将构建好的 E-R 图按照规则转化为 MySQL 能够支持的数据类型。在本系统中所需用到的数据表共有 5 个，数据表的逻辑结构设计包括的数据信息如下所示。在系统中，充分重视数据的重要性，因此在设计表结构时，均采用软删除方式，在数据库中添加 is_deleted 字段用于记录相关信息的是否删除状态，而不采取数据库中直接删除的方式。

1. 用户表（tb_bbs_user）如表 4-1 所示。

表 4-1 用户表（tb_bbs_user）

字段名	数据类型	字段大小	主/外键	NULL	说明
user_id	bigint	20	主键	N	用户主键 id
login_name	varchar	32		N	登录名称(默认为邮箱号码)
password_md5	varchar	32		N	MD5 加密后的密码
nick_name	varchar	8		N	昵称
head_img_url	varchar	128		N	头像 url
gender	varchar	4		N	性别

location	varchar	4		N	居住地
introduce	varchar	32		N	个人简介
user_status	tinyint	4		N	用户状态 0=正常 1=禁言
last_login_time	datetime	0		N	最新登录时间
create_time	datetime	0		N	注册时间

2. 分类目录表（tb_post_category）如表 4-2 所示。

表 4-2 分类目录表（tb_post_category）

字段名	数据类型	字段大小	主/外键	NULL	说明
category_id	int	11	主键	N	分类表主键
category_name	varchar	16		N	分类的名称
category_rank	int	11		N	分类的排序值被使用的越多数值越大
is_deleted	tinyint	4		N	是否删除 0=否 1=是
create_time	datetime	0		N	创建时间

3. 用户收藏表（tb_post_collect_record）如表 4-3 所示。

表 4-3 用户收藏表（tb_post_collect_record）

字段名	数据类型	字段大小	主/外键	NULL	说明
record_id	bigint	20	主键	N	主键 id
post_id	bigint	20		N	收藏帖子主键
user_id	bigint	20		N	收藏者 id
create_time	datetime	0		N	评论时间

4. 相关帖子表（tb_bbs_post）如表 4-4 所示。

表 4-4 相关帖子表（tb_bbs_post）

字段名	数据类型	字段大小	主/外键	NULL	说明
post_id	bigint	20	主键	N	帖子主键 id
publish_user_id	Bigint	20		N	发布者 id
post_title	varchar	64		N	帖子标题
post_content	mediumtex	0		N	帖子内容

	t				
post_category_id	int	11		N	帖子分类 id
post_category_name	varchar	50		N	帖子分类(冗余字段)
post_status	tinyint	4		N	0-未审核 1-审核通过 2-审核失败（默认审核通过）
post_views	bigint	20		N	阅读量
post_comments	bigint	20		N	评论量
post_collects	bigint	20		N	收藏量
last_update_time	datetime	0		N	最新修改时间
create_time	datetime	0		N	添加时间

5. 用户评论表（tb_post_comment）如表 4-5 所示。

表 4-5 用户评论表（tb_post_comment）

字段名	数据类型	字段大小	主/外键	NULL	说明
comment_id	bigint	20	主键	N	主键 id
post_id	bigint	20		N	关联的帖子主键
comment_user_id	bigint	20		N	评论者 id
comment_body	varchar	200		N	评论内容
parent_comment_user_id	bigint	20		N	所回复的上一级评论的 userId
comment_create_time	datetime	0		N	评论时间
is_deleted	tinyint	4		N	是否删除 0-未删除 1-已

4.3 数据库连接与实现

为了使 SpringBoot 框架整合数据库资源，采用 mybatis 框架用于数据持久层的操作。Mybatis 会读取其对应的配置文件，通过配置文件进行相关环境变量的配置。当相应的配置文件读取完成后，Mybatis 中将产生会话工厂即 SqlSessionFactory。通过这个会话工厂将可以创建 SqlSession 对象,它包含了运行统一数据库语句的各种方式。同时,在 MyBatis 的底层中定义了一个 executor 接口来运行数据库。它将根据通过 SqlSession 传递的参数动态地生成所有待运行的数据库语句,并承担对查询缓存的维护工作。在数据库操作中，当每个相应的数据库操作执行完成后，需要释放相关资源，以防止资源闲置而导致的内存堆积，甚至会导致栈溢出风险。但频繁的创建和释放资源会导致资源的浪费，以及系统性能的降低，因此，引入数据库池化概念，在初始化数据库连接对象时,先建立一个数据库连接池并在数据库连接池中设置特定数量的数据库执行对象，当需要执行相关操作时，从数据库连接池获取相应的初始化对象，但操作完成时，将相应的连接对象放回数据库池，等待下一个的获取操作即可。数据库连接池主要负责分配、管理并释放数据库链接,它允许应用程序中重复使用每一个现有的数据库链接,而不会再重复创建一次。释放空闲时间达到最大空闲时的数据库链接,来防止由于不能释放数据库链接所导致的数据库链接遗漏。这种技术也可以提高对数据库操作的性能。

对于主流的数据库连接池对象，C3P0 和 Druid 是被使用的较多的两款数据库连接池对象，C3P0 在功能上简单易用，稳定性高，但由于其性能较差，因此选择阿里巴巴开源的 Druid 数据库连接池对象，Druid 数据库系统连接池类型具有超强的跟踪功能,可清楚了解连接池类型和 SQL 的运行状态。另外，Druid 数据库连接池易于扩展，便于后期项目的扩展。

SpringBoot 连接数据库操作相关配置部分代码如图 4-7 所示。

```
spring:
  datasource:
    username: root
    password: root
    url: jdbc:mysql://localhost:3306/my_bbs_db?useUnicode=true&serverTimezone=Asia/Shanghai&characterEncoding=utf8
    driver-class-name: com.mysql.cj.jdbc.Driver
    type: com.alibaba.druid.pool.DruidDataSource
```

图 4-7 SpringBoot 连接数据库操作相关配置

5 系统的详细设计与实现

5.1 系统软件架构

本系统分为用户注册模块、登录模块、个人信息模块、版块管理、帖子管理、帖子回复、帖子收藏、帖子浏览、分类排行等九个模块，部分功能模块如下所示。

5.1.1 注册模块和登录模块

注册模块，用户提供信息，检验用户信息的有效性，并将用户信息持久化保存到数据库中，注册成功后可以由用户选择是否需要继续完善个人资料。对于注册模块，使用正则表达式对用户的邮箱、昵称、密码进行格式校验，对非法格式进行发现并及时处理，同时通过验证码校验的方式，防止恶意高并发请求，或脚本批量注册。对于用户密码，在后台也需要保证其隐私性，因此，当后端接收到对应密码时及时进行 MD5 加密，加密后存入数据库中。

登录模块，根据用户传入的用户名、密码以及验证码进行校验，对用户传入的密码进行 MD5 加密后与数据库中对应字段进行比较，判断用户的正确性，并将用户对应信息存入 session 中，保证用户的长久链接。同时，为保证验证码能够在各种用户状态（包括登录和未登录）下生效，将验证码存入 session 中。为实现代码的高可用性，将 session 存入 redis 中，加快其访问速度，同时减轻相应 tomcat 服务器存储 session 的压力。

针对设计隐私数据的前后端交互，本系统采用非对称加密，将用户需要传递给后端的隐私数据在前端进行加密后，通过 http 请求发送给后端，这样可以降低在发送过程中，被非法抓包，从而获取用户隐私数据的风险。

在设计注册功能的时候，为了防止通过程序手段恶意大量注册账号，在注册环节加入验证码模块，通过调用查询资料，找到适合本项目的第三方验证码模块，调用其生成验证码的方法，将生成的验证码内容以 session 作用域的方式存入 redis 中，避免使用本地内存存储验证码信息。在前端访问获取验证码对应接口时，由于浏览器自带缓存，对于同一个 url，会默认先从缓存中读取，若缓存中存在对应的内容，则不会请求对应的链接，而是将缓存中的内容返回，这样导致的结果就是验证码动态的更新，为解决该问题，引入 UUID，在调用生成验证码接口时，动态生成并拼接 UUID，使得 UUID 以 Query 参数的形式存在于链接中，这样的目的，可以达到每次访问的链接都不一样，这样就能够顺利的解决浏览器

缓存问题，而在生成 UUID 的过程中，由于 UUID 算法的的原因，导致其难以生成相同的一组 UUID，因此，可以很大程度上避免不同用户访问同一个验证码。

登录和注册功能界面图如图 5-1、5-2 所示。

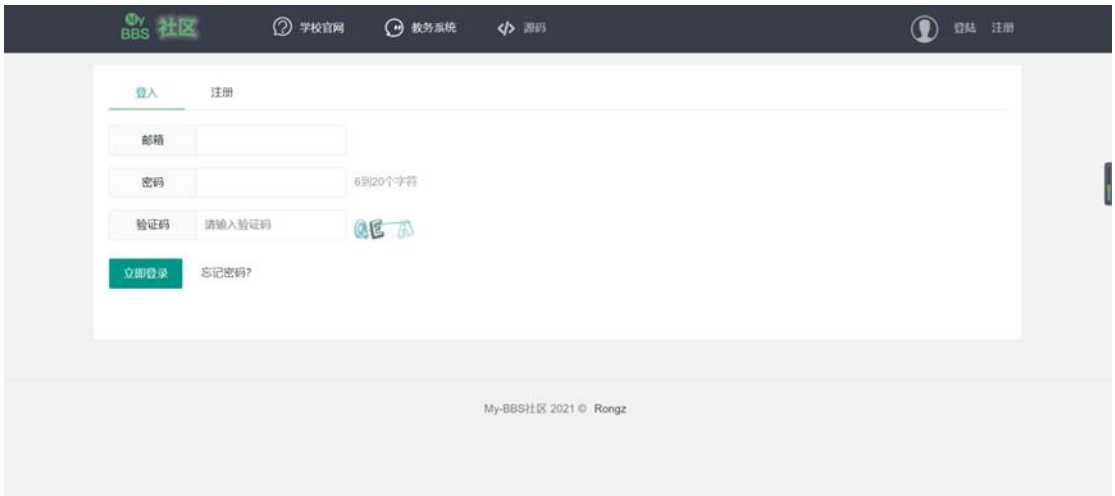

图 5-1 系统登录界面图

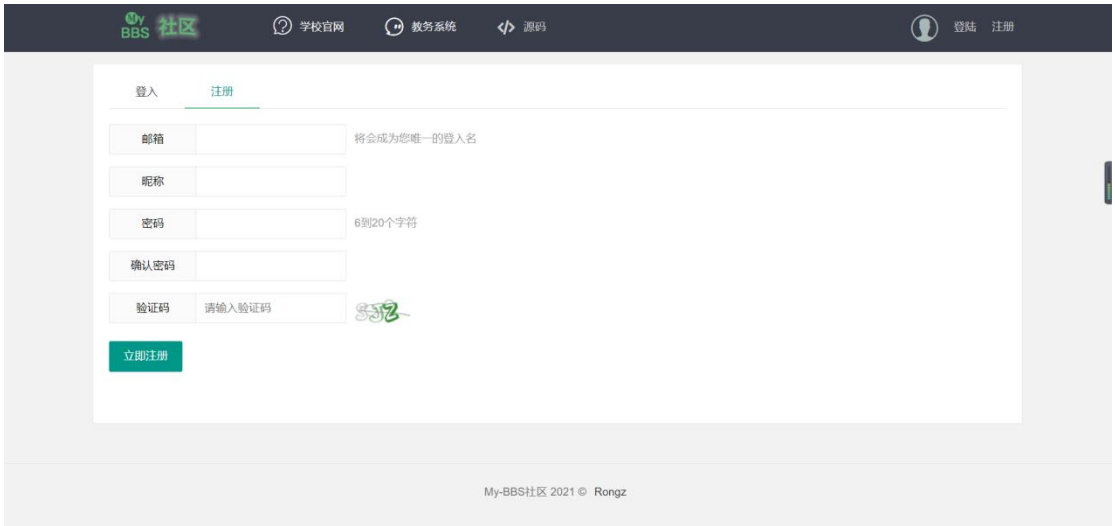

图 5-2 系统注册界面图

系统登录和注册代码如下所示。

```
@Autowired
private BBSUserMapper bbsUserMapper;

@Override
public String register(String loginName, String password, String nickName) {
    if (bbsUserMapper.selectByLoginName(loginName) != null) {
        return ServiceResultEnum.SAME_LOGIN_NAME_EXIST.getResult();
    }
    //注册用户
```

```
            BBSUser registerUser = new BBSUser();
            registerUser.setLoginName(loginName);
            registerUser.setNickName(nickName);
            //默认头像
            registerUser.setHeadImgUrl("/images/avatar/default.png");
            //默认介绍
            registerUser.setIntroduce("这个人很懒，什么都没留下~");
            //居住地
            registerUser.setLocation("未知");
            registerUser.setGender("未知");
            String passwordMD5 = MD5Util.MD5Encode(password, "UTF-8");
            registerUser.setPasswordMd5(passwordMD5);
            if (bbsUserMapper.insertSelective(registerUser) > 0) {
                return ServiceResultEnum.SUCCESS.getResult();
            }
            return ServiceResultEnum.DB_ERROR.getResult();
        }

        @Override
        public String login(String loginName, String passwordMD5, HttpSession httpSession) {
            BBSUser user = bbsUserMapper.selectByLoginNameAndPasswd(loginName,
passwordMD5);
            if (user != null && httpSession != null) {
                httpSession.setAttribute(Constants.USER_SESSION_KEY, user);
                //修改最近登录时间
                user.setLastLoginTime(new Date());
                bbsUserMapper.updateByPrimaryKeySelective(user);
                return ServiceResultEnum.SUCCESS.getResult();
            }
            return ServiceResultEnum.LOGIN_ERROR.getResult();
        }
```

5.1.2 个人信息模块

个人信息模块包括个人资料修改、查看他人信息。

个人资料修改包括相关信息包括个人基本资料、头像以及密码，需要用户在登录后，具有相应权限案后才可以修改其相关内容。对于用户密码操作，还需要对原来密码进行验证，验证通过后才可以进行修改。

其它用户可以进入对应用户的主界面进行个人信息查看，包括用户个人基本信息、用户发布的帖子以及用户发表的评论。

在设计个人资料修改模块过程中，分别采用全量更新以及局部更新的方法。全量更新，即对用户数据库中所有的字段进行更新，而局部更新采用的是对数据库部分字段进行更新，针对不同个人基本资料，包括邮箱、昵称、姓名、城市以及签名，采用全局更新的方法，而对于头像以及密码，采用的是局部更新方法。对于局部更新以及全量更新，在接口 url 设计上，采用的是同一套 url，通过参考 RESTful 风格接口设计，对于局部更新的字段采用 PATCH 请求方式，通过 PUT 请求方式对数据库字段进行全量更新。

相应功能界面如图 5-3、5-4 所示。

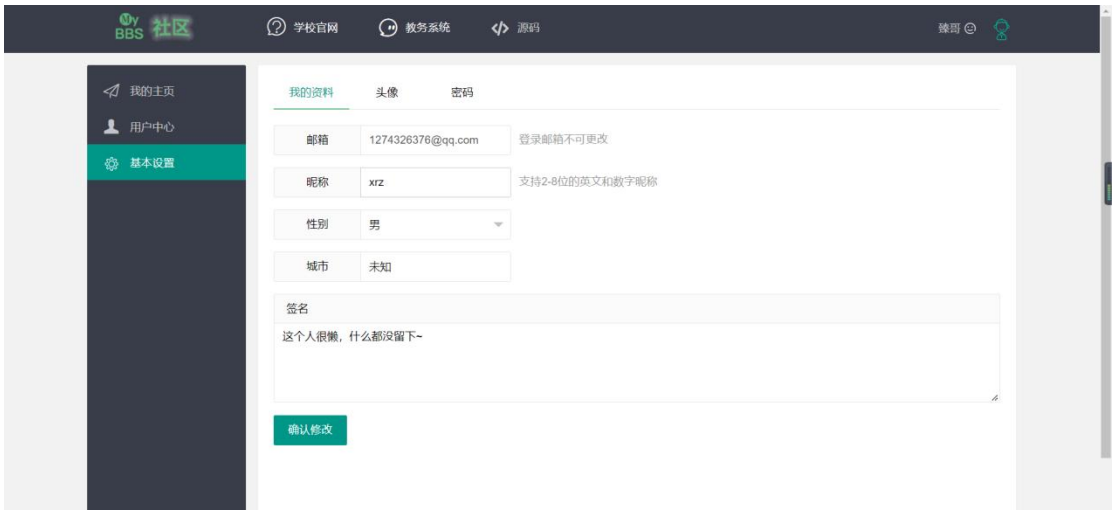

图 5-3 个人信息修改界面图

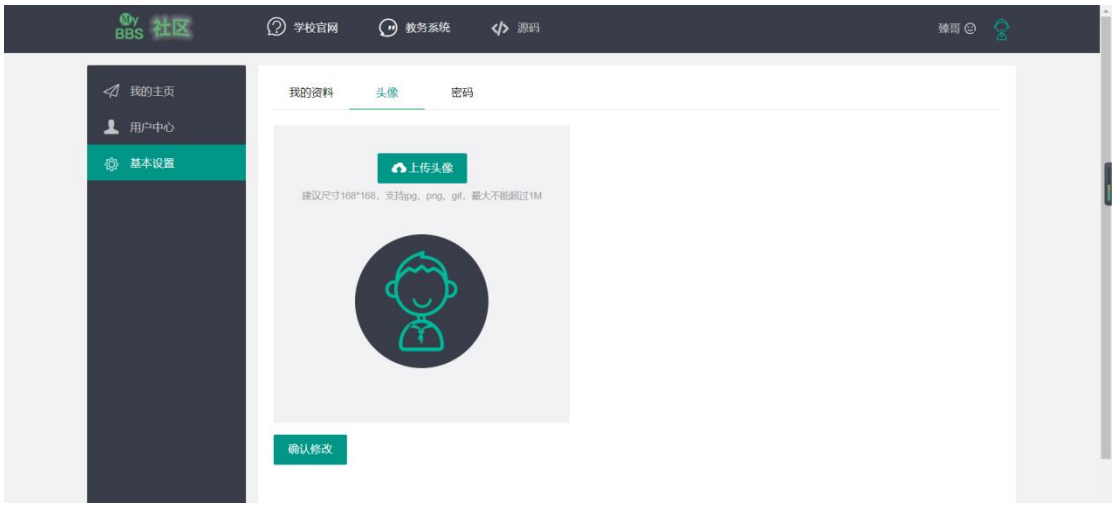

图 5-4 头像信息修改界面图

个人信息模块代码如下所示。

```
@Autowired
```

```
        private BBSUserMapper bbsUserMapper;
        public Boolean updateUserInfo(BBSUser bbsUser, HttpSession httpSession) {
    BBSUser userTemp = (BBSUser) httpSession.getAttribute(Constants.USER_SESSION_
KEY);
        BBSUser userFromDB = bbsUserMapper.selectByPrimaryKey(userTemp.getUserId
());
            //当前用户非空且状态正常才可以进行更改
            if (userFromDB != null && userFromDB.getUserStatus().intValue() == 0) {
                userFromDB.setIntroduce(SystemUtil.cleanString(bbsUser.getIntroduce()));
                userFromDB.setLocation(SystemUtil.cleanString(bbsUser.getLocation()));
                userFromDB.setGender(SystemUtil.cleanString(bbsUser.getGender()));
                userFromDB.setNickName(SystemUtil.cleanString(bbsUser.getNickName()));
                if (bbsUserMapper.updateByPrimaryKeySelective(userFromDB) > 0) {
                    httpSession.setAttribute(Constants.USER_SESSION_KEY, userFromDB);
                    return true;}}
            return false;}
        public Boolean updateUserHeadImg(BBSUser bbsUser, HttpSession httpSession) {
         BBSUser userFromDB = bbsUserMapper.selectByPrimaryKey(bbsUser.getUserId());
            //当前用户非空且状态正常才可以进行更改
            if (userFromDB != null && userFromDB.getUserStatus().intValue() == 0) {
                userFromDB.setHeadImgUrl(bbsUser.getHeadImgUrl());
                if (bbsUserMapper.updateByPrimaryKeySelective(userFromDB) > 0) {
                    httpSession.setAttribute(Constants.USER_SESSION_KEY, userFromDB);
                    return true;
                }}
            return false; }
```

5.1.3 版块管理

目前的版块管理只能由管理员在数据库中进行添加，以避免版块管理开放，而导致版块过多，难以管理以及用户筛选信息繁琐等问题。

版块管理如图 5-5 所示。

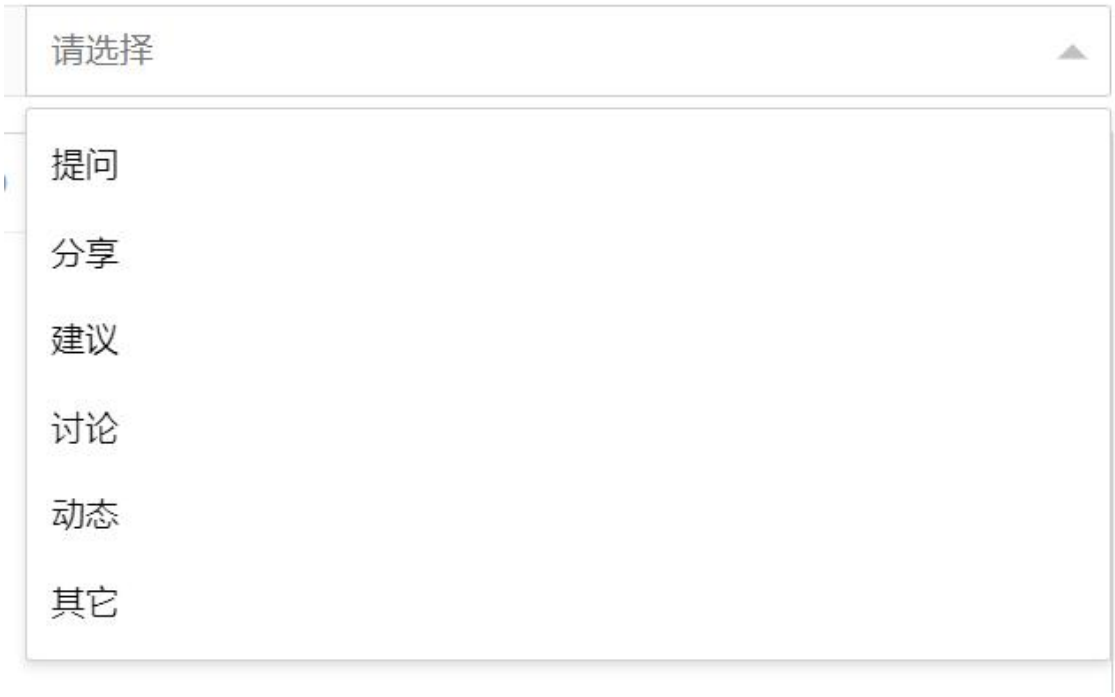

图 5-5 版块管理图

板块管理模块代码如下所示。

```
<resultMap id="BaseResultMap" type="com.my.bbs.entity.BBSPostCategory">
    <id column="category_id" jdbcType="INTEGER" property="categoryId" />
    <result  column="category_name"  jdbcType="VARCHAR"  property="categoryName"
/>
    <result column="category_rank" jdbcType="INTEGER" property="categoryRank" />
    <result column="is_deleted" jdbcType="TINYINT" property="isDeleted" />
    <result column="create_time" jdbcType="TIMESTAMP" property="createTime" />
</resultMap>
<sql id="Base_Column_List">
    category_id, category_name, category_rank, is_deleted, create_time
</sql>
<select id="getBBSPostCategories" resultMap="BaseResultMap">
    select
    <include refid="Base_Column_List" />
    from tb_post_category
    where is_deleted = 0 order by category_rank desc
</select>
```

5.1.4 帖子管理

帖子管理主要包括帖子添加、帖子内容修改更新、帖子删除。

帖子相关操作需要在用户登录的权限下进行操作，对于帖子的添加，需要绑定对应的发帖用户，以便于后期发帖人的的管理和观看者能够及时联系到发帖人。发帖人能够在个人界面查看自己发布的帖子。对于帖子的内容发布，采用富文本编辑器，使用富文本编辑器方便用户格式的管理，同时可以防范 xss 攻击。

富文本编辑器在发送相应文本内容的时候，会将对应的内容转换为 HTML 格式，而后端将接收到经 HTML 转换的存入数据库中，当下次用户从数据库中读取对应内容的时候，前端获取到对应的 HTML 内容，会自动将 HTML 内容进行转义，这样可以防范一些用户在编辑栏中写入 JavaScript 代码，从而让前端将该内容转换为 JavaScript 进行处理，给用户造成不好的使用体验。

帖子内容的更新何删除在需要验证发帖人与该操作用户对应的关系，只有操作用户就是发帖人的时候才可以进行删除和修改更新操作。

帖子管理功能界面如图 5-6、5-7 所示。

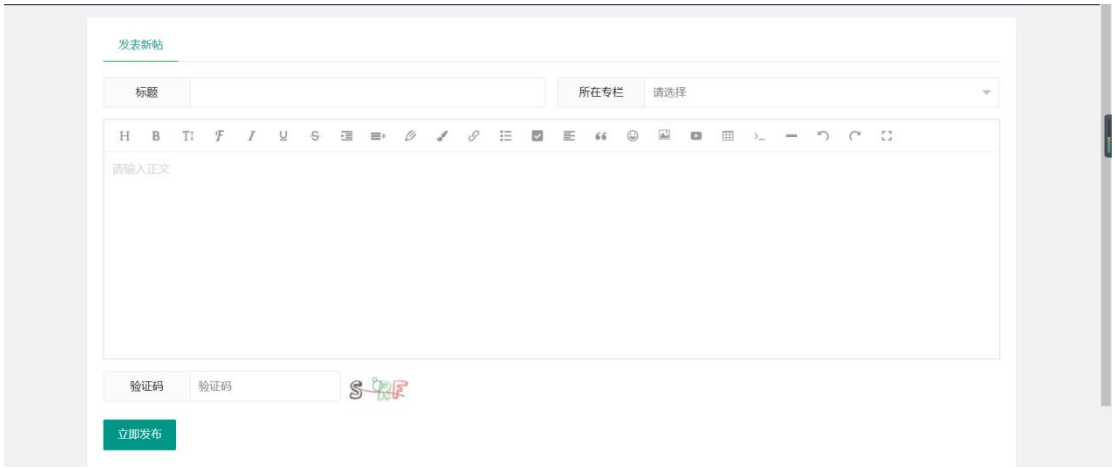

图 5-6 发布新帖图

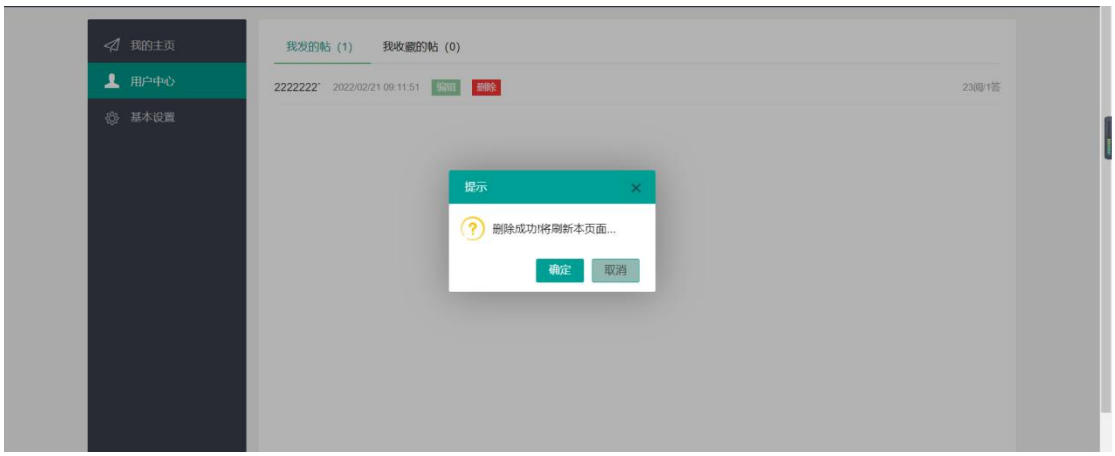

图 5-7 删除帖子图

帖子管理功能关键代码如下：

```
@PostMapping("/addPost")
```

```
        @ResponseBody
        public Result addPost(@RequestParam("postTitle") String postTitle,
                              @RequestParam("postCategoryId") Integer postCategoryId,
                              @RequestParam("postContent") String postContent,
                              @RequestParam("verifyCode") String verifyCode,
                                  HttpSession httpSession) {
            String kaptchaCode = httpSession.getAttribute(Constants.VERIFY_CODE_KEY)
+ "";
            if (!StringUtils.hasLength(kaptchaCode) || !verifyCode.equalsIgnoreCase(kaptchaC
ode)) {return ResultGenerator.genFailResult(ServiceResultEnum.LOGIN_VERIFY_CODE_ER
ROR.getResult());}
            BBSUser bbsUser = (BBSUser) httpSession.getAttribute(Constants.USER_SESSI
ON_KEY);
            BBSPost bbsPost = new BBSPost();
            bbsPost.setPublishUserId(bbsUser.getUserId());
            bbsPost.setPostTitle(postTitle);
            bbsPost.setPostContent(postContent);
            bbsPost.setPostCategoryId(postCategoryId);
            bbsPost.setPostCategoryName(bbsPostCategory.getCategoryName());
            if (bbsPostService.savePost(bbsPost) > 0) {
                httpSession.removeAttribute(Constants.VERIFY_CODE_KEY);//清空 session
中的验证码信息
                return ResultGenerator.genSuccessResult();
            } else {
                return ResultGenerator.genFailResult("请求失败，请检查参数及账号是否有操
作权限");}}
```

5.1.5 帖子收藏、回复和浏览以及排行榜

帖子的收藏只需要相关用户点击收藏按钮即可，用户可以在个人主页查看自己相应的收藏记录。

帖子回复包括分级回复，第一级回复的就是发帖人，二级回复所回复的是一级发帖人，以此类推，在回复的时候，数据库中会记录相应的信息和上一级回复人，便于前端的展示。

当用户点击并进入相应帖子的时候，相应的帖子浏览量即可加一，通过浏览量便于后期排行榜的操作。

排行榜根据浏览量分为默认、周榜、月榜，同时根据热度和最新进行排行。

周榜、月榜是通过本周、本月的浏览量进行升序降序操作，默认数据是对数据库中所有的帖子根据浏览量进行排行，当浏览量相同的时候，对收藏量进行比较并排行。热度和最新分别根据浏览量、更新时间对数据库中的所有数据进行排序。

相应功能界面如图 5-8、5-9 所示。

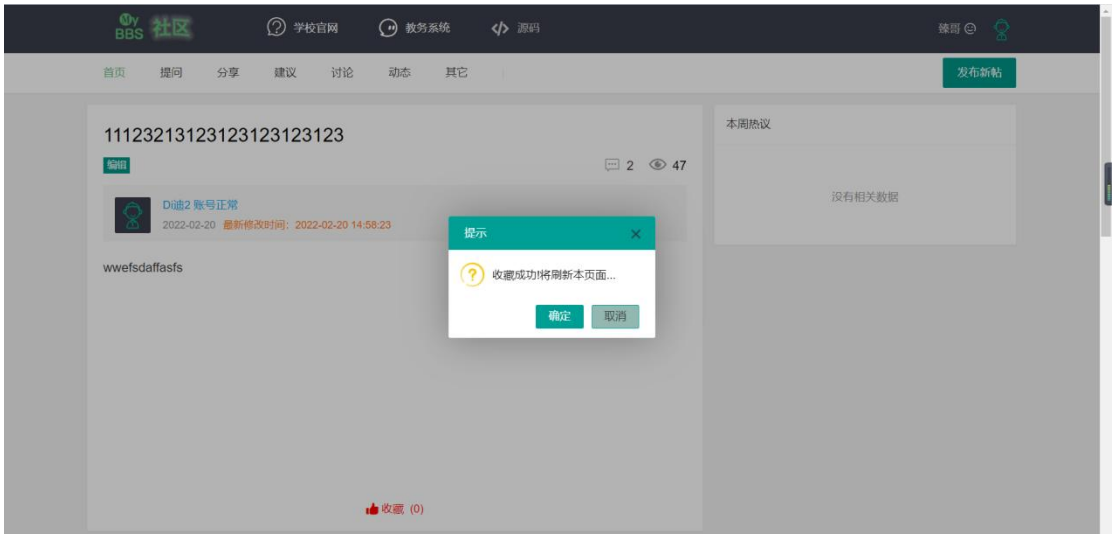

图 5-8 帖子收藏图

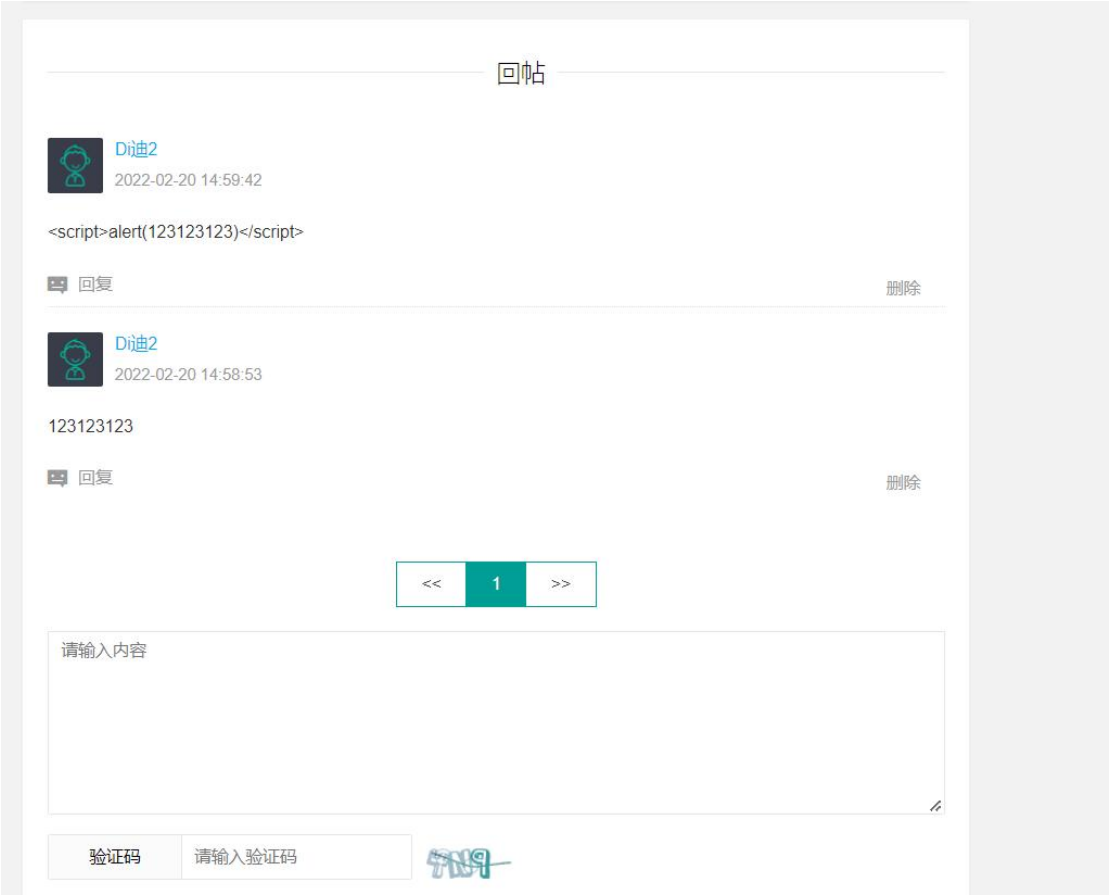

图 5-9 帖子评论-回复图

帖子模块相关关键代码如下所示。

```
        @PostMapping("/replyPost")
        @ResponseBody
        public Result replyPost(@RequestParam("postId") Long postId,
                                @RequestParam(value = "parentCommentUserId", required
= false) Long parentCommentUserId,
                                @RequestParam("commentBody") String commentBody,
                                @RequestParam("verifyCode") String verifyCode,
                                HttpSession httpSession) {
            String kaptchaCode = httpSession.getAttribute(Constants.VERIFY_CODE_KEY)
 + "";
            if (!StringUtils.hasLength(kaptchaCode) || !verifyCode.equalsIgnoreCase(kaptcha
Code)) {return ResultGenerator.genFailResult(ServiceResultEnum.LOGIN_VERIFY_CODE_E
RROR.getResult());}
            BBSUser bbsUser = (BBSUser) httpSession.getAttribute(Constants.USER_SES
SION_KEY);
            BBSPostComment bbsPostComment = new BBSPostComment();
            bbsPostComment.setCommentBody(commentBody);
            bbsPostComment.setCommentUserId(bbsUser.getUserId());
            bbsPostComment.setParentCommentUserId(parentCommentUserId);
            bbsPostComment.setPostId(postId);
            if (bbsPostCommentService.addPostComment(bbsPostComment)) {
                httpSession.removeAttribute(Constants.VERIFY_CODE_KEY);
                return ResultGenerator.genSuccessResult();
            } else {
                return ResultGenerator.genFailResult("请求失败，请检查参数及账号是否有
操作权限");}}
```

6 系统部署与测试

6.1 系统测试

6.1.1 系统测试目的

软件的系统测试是发现系统开发过程中遗留的错误，导致系统异常运行的各种错误和缺陷。系统测试应根据系统开发过程的每个阶段的要求设计不同的测试示例，并使用这些不同的示例对系统进行测试。使用不同的示例对系统进行测试可以更有效的去发现系统存在的问题，减少系统投入使用时出现的问题[19]。

6.1.2 系统测试过程与结果

（1）用户注册和登录功能：本系统采用 Ajax 对注册字段进行校验，当点击“立即注册”按钮后，就会对相应输入框的字段进行格式校验，若相应输入框的内容不符合要求，将会弹出相应提示，如果符合均条件，调用后端相应注册接口。测试效过如图 6-1、6-2、6-3 所示。

图 6-1 用户注册测试页面

图 6-2 用户登录测试页面

当输入符合系统要求的信息，注册功能是比较完善的，没有发现问题。注册效果如下所示。

图 6-3 用户登录成功测试页面

用户注册功能测试成功，然后开始测试用户登录功能，当用户名和密码输入正确后，会跳转进主页面，若用户名或者密码输入错误，会显示登录失败弹窗。测试结果可以登录，功能已实现。登录失败功能效果如图 6-4 所示。

图 6-4 用户登录失败测试页面

（2）帖子发表和帖子查看功能：用户输入相应标题后，选择相应专栏，输

入正文内容和验证码后，即可实现发帖功能。用户进入主界面后可以所有发布的帖子。测试结果，可以成功发帖，并在主界面可以看到相应的新帖子，以及之前所发的帖子。

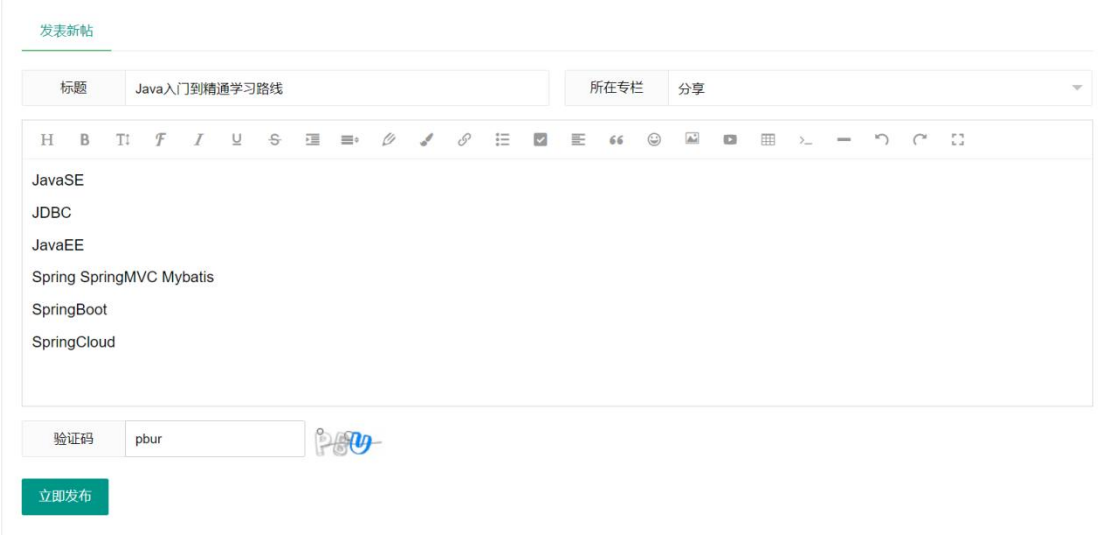

图 6-5 用户发表新帖测试页面

图 6-6 用户发表成功测试页面

由于功能模块较多，只对系统的用户管理模块进行测试结果书写，其他功能模块的测试结果省略。

6.2 系统部署

系统部署是指将系统部署到远端 Linux 服务器上，让用户可以通过域名或 ip 进行直接访问，而不需要局限在局域网中，给予用户更好的体验。

6.3 部署过程遇到的问题和解决方法

在系统部署过程中，发现系统无法支撑高并发以及系统安全性较低。系统部署远端服务器后，使用 python 脚本进行高并发访问，发现在单节点服务器情况下，该系统仅能支撑约 100 个高并发同时访问，对于多用户高并发请求无法给予用户良好的体验。同时，由于裸露服务器 ip 地址，导致系统安全性较低，裸露的 ip 容易被他人进行攻击。对于该问题，在部署过程中引入 Nginx，Nginx 通过反向代理以及负载均衡可以很好的解决系统安全性问题，同时，Nginx 的负载均衡算法[20]，使得一台服务器中的系统可以将并发量提高至 500。

系统部署架构图如图 6-7 所示。

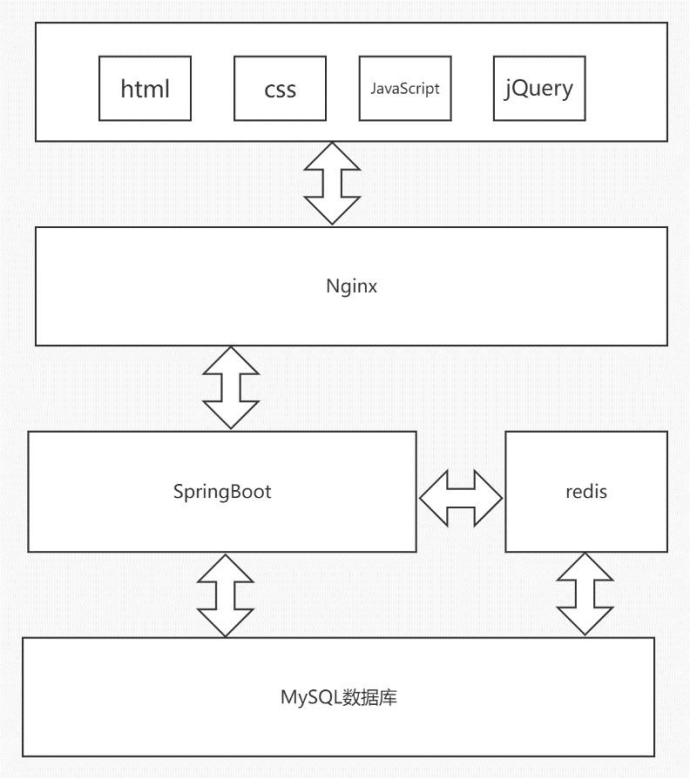

图 6-7 系统部署架构图

6.4 系统存在的不足

经过反复修改和测试，本系统已完成了在需求分析中的各项功能，基本满足了校园论坛的基本需求，但由于时间有限以及个人开发经验的欠缺，该系统还存在以下不足：

1、对于验证码校验，后台服务器仅能辨别用户输入小写验证码，而无法实现不区分大小写的功能，因此，在后台验证码校验过程中，加入忽略大小写比较，

给与用户良好的体验

2、对于注册成功的用户，前端无法感知是否注册成功，以至于无法给予用户及时的反馈，因此，在用户注册成功后，由后端返回成功的消息，由前端感知后，将用户跳转到登录界面。

7 结语

本文对校园论坛的研究背景、现状进行了全面的调研和分析，并在收集闽江学院学生群体建议和意见的基础上提出了校园论坛系统需求，完成了系统设计、系统开发、系统实现和测试工作。论文的主要工作包括:

第一，全面地阐述了校园论坛的研究背景与研究意义，并对系统进行了需求分析、可行性分析与模块分析。

第二，设计实现了注册登录、板块管理、帖子管理、排行榜等功能模块，并通过测试。

校园论坛系统已经能够满足基本的操作需求，达到预期的效果，但也还存在着一些不足，如：缺少管理员后台系统、缺少搜索功能等，这将在以后的二次开发中进一步予以完善。

参考文献

[1] 江奇峰.基于 Java 数据库编程及其应用探究[J].电脑编程技巧与维护,2021(04):15-16+57.

[2] 孙辉中.JAVA 编程语言在计算机软件开发中的应用[J].网络安全技术与应用,2022(01):49-50.

[3] Understanding LINUX network internals[J]. Choice Reviews Online,2006,44(01).

[4] 孔美云,钱成特,刘加海. Linux 程序在 Windows 上的移植及运行稳定性分析[J]. 浙江大学学报(理学版),2012,39(03):289-292.

[5] 李巍巍,扈先勤. Linux 服务器安全防护部署[J]. 科技传播,2014,6(24):141-143.

[6] Andrew Blaich. Reflections on the virtues of modularity: a case study in linux security modules[J]. Software: Practice and experience,2009,39(15).

[7] 侯红君. Linux 0.11 内核研究与内核编程实践[D]. 天津大学,2007.

[8] 董军. Linux 内核开发者协作模式研究[D]. 大连理工大学,2016.

[9] Bernstein D . Containers and Cloud: From LXC to Docker to Kubernetes[J]. Cloud Computing, IEEE, 2014, 1(3):81-84.

[10] 张建,谢天钧. 基于 Docker 的平台即服务架构研究[J]. 信息技术与信息化,2014(10):131-134.

[11] 喻佳,吴丹新.基于 SpringBoot 的 Web 快速开发框架[J].电脑编程技巧与维护,2021(09):31-33.

[12] 邹红霆.基于 SSM 框架的 Web 系统研究与应用[J].湖南理工学院学报(自然科学版),2017,30(01):39-43.

[13] Anonymous. Cacheonix Releases Distributed MyBatis Cache[J]. Wireless News,2011.

[14] 陈立兵.IntelliJ IDEA——开发人员利器[J].程序员,2009(01):118-120.

[15] 宣振国. 基于 Mysql 的数据库集群设计与实现[D].北京邮电大学,2013.

[16] 曾超宇,李金香.Redis 在高速缓存系统中的应用[J].微型机与应用,2013,32(12):11-13.

[17] 邱祝文.基于 redis 的分布式缓存系统架构研究[J].网络安全技术与应用,2014(10):52+54.

[18] 任中方,张华,闫明松,陈世福.MVC 模式研究的综述[J].计算机应用研究,2004(10):1-4+8.

[19] 宋俊雅,王鹏彪,黄俊爽,郭凌云.B/S 结构软件的系统测试技术[J].科技信息,2010(10):247-248.

[20] 林丽丽.使用高性能 Web 服务器 Nginx 实现开源负载均衡[J].大众科技,2010(07):37-38+27.

致谢

首先感谢学校给了我们学习的机会，让我们在优越的条件下，完成了学业。不仅如此，在这里，我还认识了一群可爱的人，是他们，让我的大学生活变得如此的丰富多彩，成为我人生忠告难以忘怀的回忆。感谢他们，在我学习生活中无私的给予的帮助。感谢林苗老师从选题指导、论文框架到细节修改，都给予了细致的指导，提出了很多宝贵的意见与建议。

最后，感谢我的父母，二十余载的关怀和照顾，让家成为我最温暖的避风港，正是有了你们的支持，我才能读完大学，让我有信心完成毕业论文的写作。

时光匆匆如流水，转眼便是大学毕业时节，春梦秋云，聚散真容易。离校日期已日趋临近，毕业论文的的完成也随之进入了尾声。

从开始进入课题到论文的顺利完成，一直都离不开老师、同学、朋友给我热情的帮助，还有许多人，也许他们只是我生命中匆匆的过客，但他们对我的支持和帮助依然在我记忆中留底了深刻的印象。在此无法一一罗列，但对他们，我始终心怀感激。

感谢所有在我写作期间帮助我的老师、同学、父母、朋友，你们的帮助对我完成论文是不可或缺的。对此，我永远铭记在心！

附录

```
@Controller
public class CaptchaController {

    @GetMapping("/common/captcha")
    public void defaultKaptcha(HttpServletRequest httpServletRequest, HttpServletResponse
httpServletResponse) throws Exception {
        httpServletResponse.setHeader("Cache-Control", "no-store");
        httpServletResponse.setHeader("Pragma", "no-cache");
        httpServletResponse.setDateHeader("Expires", 0);
        httpServletResponse.setContentType("image/gif");
        // 三个参数分别为宽、高、位数
        GifCaptcha captcha = new GifCaptcha(75, 30,4);
        // 设置类型 数字和字母混合
        captcha.setCharType(Captcha.TYPE_DEFAULT);
        //设置字体
        captcha.setCharType(Captcha.FONT_9);
        // 验证码存入 session
    httpServletRequest.getSession().setAttribute(Constants.VERIFY_CODE_KEY,
captcha.text().toLowerCase());
        // 输出图片流
        captcha.out(httpServletResponse.getOutputStream());
    }
}

@Controller
public class ErrorPageController implements ErrorController {

    private static ErrorPageController errorPageController;

    @Autowired
    private ErrorAttributes errorAttributes;

    private final static String ERROR_PATH = "/error";

    public ErrorPageController(ErrorAttributes errorAttributes) {
```

```
        this.errorAttributes = errorAttributes;
    }

    public ErrorPageController() {
        if (errorPageController == null) {
            errorPageController = new ErrorPageController(errorAttributes);
        }
    }

    @RequestMapping(value = ERROR_PATH, produces = "text/html")
    public ModelAndView errorHtml(HttpServletRequest request) {
        HttpStatus status = getStatus(request);
        if (HttpStatus.BAD_REQUEST == status) {
            return new ModelAndView("error/error_400");
        } else if (HttpStatus.NOT_FOUND == status) {
            return new ModelAndView("error/error_404");
        } else {
            return new ModelAndView("error/error_5xx");
        }
    }

    @RequestMapping(value = ERROR_PATH)
    @ResponseBody
    public ResponseEntity<Map<String, Object>> error(HttpServletRequest request) {
        Map<String, Object> body = getErrorAttributes(request, getTraceParameter(request));
        HttpStatus status = getStatus(request);
        return new ResponseEntity<Map<String, Object>>(body, status);
    }

    @Override
    public String getErrorPath() {
        return ERROR_PATH;
    }


    private boolean getTraceParameter(HttpServletRequest request) {
        String parameter = request.getParameter("trace");
        if (parameter == null) {
```

```
            return false;
        }
        return !"false".equals(parameter.toLowerCase());
    }

    protected Map<String, Object> getErrorAttributes(HttpServletRequest request, boolean includeStackTrace) {
        WebRequest webRequest = new ServletWebRequest(request);
        return this.errorAttributes.getErrorAttributes(webRequest, includeStackTrace);
    }

    private HttpStatus getStatus(HttpServletRequest request) {
        Integer statusCode = (Integer) request
                .getAttribute("javax.servlet.error.status_code");
        if (statusCode != null) {
            try {
                return HttpStatus.valueOf(statusCode);
            } catch (Exception ex) {
            }
        }
        return HttpStatus.INTERNAL_SERVER_ERROR;
    }
}

@RestControllerAdvice
public class MyBBSExceptionHandler {

    @ExceptionHandler(Exception.class)
    public Object handleException(Exception e, HttpServletRequest req) {
        Result result = new Result();
        result.setResultCode(500);

        e.printStackTrace();
        result.setMessage("未知异常，请联系管理员");
        //检查请求是否为 ajax，如果是 ajax 请求则返回 Result json 串，如果不是 ajax 请求则返回 error 视图
        String contentTypeHeader = req.getHeader("Content-Type");
        String acceptHeader = req.getHeader("Accept");
```

```
            String xRequestedWith = req.getHeader("X-Requested-With");
            if ((contentTypeHeader != null && contentTypeHeader.contains("application/json"))
                    || (acceptHeader != null && acceptHeader.contains("application/json"))
                    || "XMLHttpRequest".equalsIgnoreCase(xRequestedWith)) {
                return result;
            } else {
                ModelAndView modelAndView = new ModelAndView();
                modelAndView.addObject("message", e.getMessage());
                modelAndView.addObject("url", req.getRequestURL());
                modelAndView.addObject("stackTrace", e.getStackTrace());
                modelAndView.addObject("author", "十三");
                modelAndView.addObject("ltd", "My BBS");
                modelAndView.setViewName("error");
                return modelAndView;
            }
        }
}

@Controller
public class UploadController {

    @PostMapping({"/uploadFile"})
    @ResponseBody
    public Result upload(HttpServletRequest httpServletRequest, @RequestParam("file")
MultipartFile file) throws URISyntaxException {
            String fileName = file.getOriginalFilename();
            String suffixName = fileName.substring(fileName.lastIndexOf("."));
            //生成文件名称通用方法
            SimpleDateFormat sdf = new SimpleDateFormat("yyyyMMdd_HHmmss");
            Random r = new Random();
            StringBuilder tempName = new StringBuilder();
            tempName.append(sdf.format(new
Date())).append(r.nextInt(100)).append(suffixName);
            String newFileName = tempName.toString();
            File fileDirectory = new File(Constants.FILE_UPLOAD_DIC);
            //创建文件
            File destFile = new File(Constants.FILE_UPLOAD_DIC + newFileName);
            try {
```

```
            if (!fileDirectory.exists()) {
                if (!fileDirectory.mkdir()) {
                    throw new IOException("文件夹创建失败,路径为：" + fileDirectory);
                }
            }
            file.transferTo(destFile);
            Result resultSuccess = ResultGenerator.genSuccessResult();
            resultSuccess.setData(SystemUtil.getHost(new
URI(httpServletRequest.getRequestURL() + "")) + "/upload/" + newFileName);
            return resultSuccess;
        } catch (IOException e) {
            e.printStackTrace();
            return ResultGenerator.genFailResult("文件上传失败");
        }
    }

    @PostMapping({"/uploadFiles"})
    @ResponseBody
    public Result uploadV2(HttpServletRequest httpServletRequest) throws
URISyntaxException {
        List<MultipartFile> multipartFiles = new ArrayList<>(8);
        CommonsMultipartResolver multipartResolver = new
CommonsMultipartResolver(httpServletRequest.getSession().getServletContext());
        if (multipartResolver.isMultipart(httpServletRequest)) {
            MultipartHttpServletRequest multiRequest = (MultipartHttpServletRequest)
httpServletRequest;
            Iterator<String> iter = multiRequest.getFileNames();
            int total = 0;
            while (iter.hasNext()) {
                if (total > 5) {
                    return ResultGenerator.genFailResult("最多上传 5 张图片");
                }
                total += 1;
                MultipartFile file = multiRequest.getFile(iter.next());
                multipartFiles.add(file);
            }
        }
        if (CollectionUtils.isEmpty(multipartFiles)) {
```

```
            return ResultGenerator.genFailResult("参数异常");
        }
        if (multipartFiles != null && multipartFiles.size() > 5) {
            return ResultGenerator.genFailResult("最多上传 5 张图片");
        }
        List<String> fileNames = new ArrayList(multipartFiles.size());
        for (int i = 0; i < multipartFiles.size(); i++) {
            String fileName = multipartFiles.get(i).getOriginalFilename();
            String suffixName = fileName.substring(fileName.lastIndexOf("."));
            //生成文件名称通用方法
            SimpleDateFormat sdf = new SimpleDateFormat("yyyyMMdd_HHmmss");
            Random r = new Random();
            StringBuilder tempName = new StringBuilder();
            tempName.append(sdf.format(new
Date())).append(r.nextInt(100)).append(suffixName);
            String newFileName = tempName.toString();
            File fileDirectory = new File(Constants.FILE_UPLOAD_DIC);
            //创建文件
            File destFile = new File(Constants.FILE_UPLOAD_DIC + newFileName);
            try {
                if (!fileDirectory.exists()) {
                    if (!fileDirectory.mkdir()) {
                        throw new IOException("文件夹创建失败,路径为：" +
fileDirectory);
                    }
                }
                multipartFiles.get(i).transferTo(destFile);
                fileNames.add(SystemUtil.getHost(new
URI(httpServletRequest.getRequestURL() + "")) + "/upload/" + newFileName);
            } catch (IOException e) {
                e.printStackTrace();
                return ResultGenerator.genFailResult("文件上传失败");
            }
        }
        Result resultSuccess = ResultGenerator.genSuccessResult();
        resultSuccess.setData(fileNames);
        return resultSuccess;
    }
```

```
}

@Controller
public class BBSPostCollectController {

    @Resource
    private BBSPostCollectService bbsPostCollectService;

    @PostMapping("/addCollect/{postId}")
    @ResponseBody
    public Result addCollect(@PathVariable("postId") Long postId,
                             HttpSession httpSession) {
        if (null == postId || postId < 0) {
            return ResultGenerator.genFailResult("postId 参数错误");
        }
        BBSUser bbsUser = (BBSUser)
httpSession.getAttribute(Constants.USER_SESSION_KEY);
        if (bbsPostCollectService.addCollectRecord(bbsUser.getUserId(), postId)) {
            return ResultGenerator.genSuccessResult();
        } else {
            return ResultGenerator.genFailResult("请求失败，请检查参数及账号是否有操作
权限");
        }
    }

    @PostMapping("/delCollect/{postId}")
    @ResponseBody
    public Result delCollect(@PathVariable("postId") Long postId,
                             HttpSession httpSession) {
        if (null == postId || postId < 0) {
            return ResultGenerator.genFailResult("postId 参数错误");
        }
        BBSUser bbsUser = (BBSUser)
httpSession.getAttribute(Constants.USER_SESSION_KEY);
        if (bbsPostCollectService.deleteCollectRecord(bbsUser.getUserId(), postId)) {
            return ResultGenerator.genSuccessResult();
        } else {
```

```
                return ResultGenerator.genFailResult("请求失败，请检查参数及账号是否有操作
权限");
            }
        }
}

@Controller
public class BBSPostCommentController {

    @Resource
    private BBSPostCommentService bbsPostCommentService;

    @PostMapping("/replyPost")
    @ResponseBody
    public Result replyPost(@RequestParam("postId") Long postId,
                            @RequestParam(value = "parentCommentUserId", required =
false) Long parentCommentUserId,
                            @RequestParam("commentBody") String commentBody,
                            @RequestParam("verifyCode") String verifyCode,
                            HttpSession httpSession) {
        if (null == postId || postId < 0) {
            return ResultGenerator.genFailResult("postId 参数错误");
        }
        if (!StringUtils.hasLength(commentBody)) {
            return ResultGenerator.genFailResult("commentBody 参数错误");
        }
        if (commentBody.trim().length() > 200) {
            return ResultGenerator.genFailResult("评论内容过长");
        }
        String kaptchaCode = httpSession.getAttribute(Constants.VERIFY_CODE_KEY) + "";
        if (!StringUtils.hasLength(kaptchaCode) || !verifyCode.equalsIgnoreCase(kaptchaCode))
{
            return
ResultGenerator.genFailResult(ServiceResultEnum.LOGIN_VERIFY_CODE_ERROR.getResult()
);
        }
        BBSUser bbsUser = (BBSUser)
httpSession.getAttribute(Constants.USER_SESSION_KEY);
```

```
        BBSPostComment bbsPostComment = new BBSPostComment();
        bbsPostComment.setCommentBody(commentBody);
        bbsPostComment.setCommentUserId(bbsUser.getUserId());
        bbsPostComment.setParentCommentUserId(parentCommentUserId);
        bbsPostComment.setPostId(postId);

        if (bbsPostCommentService.addPostComment(bbsPostComment)) {
            httpSession.removeAttribute(Constants.VERIFY_CODE_KEY);
            return ResultGenerator.genSuccessResult();
        } else {
            return ResultGenerator.genFailResult("请求失败，请检查参数及账号是否有操作
权限");
        }
    }

    @PostMapping("/delReply/{commentId}")
    @ResponseBody
    public Result delReply(@PathVariable("commentId") Long commentId,
                           HttpSession httpSession) {
        if (null == commentId || commentId < 0) {
            return ResultGenerator.genFailResult("commentId 参数错误");
        }
        BBSUser bbsUser = (BBSUser)
httpSession.getAttribute(Constants.USER_SESSION_KEY);
        if (bbsPostCommentService.delPostComment(commentId,bbsUser.getUserId())) {
            return ResultGenerator.genSuccessResult();
        } else {
            return ResultGenerator.genFailResult("请求失败，请检查参数及账号是否有操作
权限");
        }
    }
}

@Controller
public class BBSPostController {
    @Resource
    private BBSPostService bbsPostService;
```

```
    @Resource
    private BBSPostCategoryService bbsPostCategoryService;
    @Resource
    private BBSUserService bbsUserService;
    @Resource
    private BBSPostCollectService bbsPostCollectService;
    @Resource
    private BBSPostCommentService bbsPostCommentService;

    @GetMapping("detail/{postId}")
    public String postDetail(HttpServletRequest request, @PathVariable(value = "postId") Long postId,
                                @RequestParam(value = "commentPage", required = false, defaultValue = "1") Integer commentPage) {
        List<BBSPostCategory> bbsPostCategories = bbsPostCategoryService.getBBSPostCategories();
        if (CollectionUtils.isEmpty(bbsPostCategories)) {
            return "error/error_404";
        }
        //将分类数据封装到 request 域中
        request.setAttribute("bbsPostCategories", bbsPostCategories);
        // 帖子内容
        BBSPost bbsPost = bbsPostService.getBBSPostForDetail(postId);
        if (bbsPost == null) {
            return "error/error_404";
        }
        request.setAttribute("bbsPost", bbsPost);
        // 发帖用户信息
        BBSUser bbsUser = bbsUserService.getUserById(bbsPost.getPublishUserId());
        if (bbsUser == null) {
            return "error/error_404";
        }
        request.setAttribute("bbsUser", bbsUser);
        // 是否收藏了本贴
        BBSUser currentUser = (BBSUser) request.getSession().getAttribute(Constants.USER_SESSION_KEY);
        request.setAttribute("currentUserCollectFlag", bbsPostCollectService.validUserCollect(currentUser.getUserId(), postId));
```

```
        // 本周热议的帖子
        request.setAttribute("hotTopicBBSPostList",
bbsPostService.getHotTopicBBSPostList());
        // 评论数据
        PageResult commentsPage = bbsPostCommentService.getCommentsByPostId(postId,
commentPage);
        request.setAttribute("commentsPage", commentsPage);
        return "jie/detail";
    }

    @GetMapping("editPostPage/{postId}")
    public String editPostPage(HttpServletRequest request, @PathVariable(value = "postId")
Long postId) {
        BBSUser bbsUser = (BBSUser)
request.getSession().getAttribute(Constants.USER_SESSION_KEY);
        List<BBSPostCategory> bbsPostCategories =
bbsPostCategoryService.getBBSPostCategories();
        if (CollectionUtils.isEmpty(bbsPostCategories)) {
            return "error/error_404";
        }
        //将分类数据封装到 request 域中
        request.setAttribute("bbsPostCategories", bbsPostCategories);
        if (null == postId || postId < 0) {
            return "error/error_404";
        }
        BBSPost bbsPost = bbsPostService.getBBSPostById(postId);
        if (bbsPost == null) {
            return "error/error_404";
        }
        if (!bbsUser.getUserId().equals(bbsPost.getPublishUserId())) {
            request.setAttribute("message", "非本人发帖，无权限修改");
            return "error/error";
        }
        request.setAttribute("bbsPost", bbsPost);
        request.setAttribute("postId", postId);
        return "jie/edit";
    }
```

```
    @GetMapping("/addPostPage")
    public String addPostPage(HttpServletRequest request) {
        List<BBSPostCategory>                    bbsPostCategories                    =
bbsPostCategoryService.getBBSPostCategories();
        if (CollectionUtils.isEmpty(bbsPostCategories)) {
            return "error/error_404";
        }
        //将分类数据封装到 request 域中
        request.setAttribute("bbsPostCategories", bbsPostCategories);
        return "jie/add";
    }

    @PostMapping("/addPost")
    @ResponseBody
    public Result addPost(@RequestParam("postTitle") String postTitle,
                          @RequestParam("postCategoryId") Integer postCategoryId,
                          @RequestParam("postContent") String postContent,
                          @RequestParam("verifyCode") String verifyCode,
                          HttpSession httpSession) {
        if (!StringUtils.hasLength(postTitle)) {
            return ResultGenerator.genFailResult("postTitle 参数错误");
        }
        if (null == postCategoryId || postCategoryId < 0) {
            return ResultGenerator.genFailResult("postCategoryId 参数错误");
        }
        BBSPostCategory                    bbsPostCategory                    =
bbsPostCategoryService.selectById(postCategoryId);
        if (null == bbsPostCategory) {
            return ResultGenerator.genFailResult("postCategoryId 参数错误");
        }
        if (!StringUtils.hasLength(postContent)) {
            return ResultGenerator.genFailResult("postContent 参数错误");
        }
        if (postTitle.trim().length() > 32) {
            return ResultGenerator.genFailResult("标题过长");
        }
        if (postContent.trim().length() > 100000) {
            return ResultGenerator.genFailResult("内容过长");
```

```
        }
        String kaptchaCode = httpSession.getAttribute(Constants.VERIFY_CODE_KEY) + "";
        if (!StringUtils.hasLength(kaptchaCode) || !verifyCode.equalsIgnoreCase(kaptchaCode))
{
            return
ResultGenerator.genFailResult(ServiceResultEnum.LOGIN_VERIFY_CODE_ERROR.getResult()
);
        }
        BBSUser bbsUser = (BBSUser)
httpSession.getAttribute(Constants.USER_SESSION_KEY);
        BBSPost bbsPost = new BBSPost();
        bbsPost.setPublishUserId(bbsUser.getUserId());
        bbsPost.setPostTitle(postTitle);
        bbsPost.setPostContent(postContent);
        bbsPost.setPostCategoryId(postCategoryId);
      bbsPost.setPostCategoryName(bbsPostCategory.getCategoryName());
        if (bbsPostService.savePost(bbsPost) > 0) {
            httpSession.removeAttribute(Constants.VERIFY_CODE_KEY);//清空 session 中
的验证码信息
            return ResultGenerator.genSuccessResult();
        } else {
            return ResultGenerator.genFailResult("请求失败，请检查参数及账号是否有操作
权限");
        }
    }

    @PostMapping("/delPost/{postId}")
    @ResponseBody
    public Result delPost(@PathVariable("postId") Long postId,
                          HttpSession httpSession) {
        if (null == postId || postId < 0) {
            return ResultGenerator.genFailResult("postId 参数错误");
        }
        BBSUser bbsUser = (BBSUser)
httpSession.getAttribute(Constants.USER_SESSION_KEY);
        if (bbsPostService.delBBSPost(bbsUser.getUserId(), postId) > 0) {
            return ResultGenerator.genSuccessResult();
        } else {
```

```
            return ResultGenerator.genFailResult("请求失败，请检查参数及账号是否有操作权限");
        }
    }

    @PostMapping("/editPost")
    @ResponseBody
    public Result editPost(@RequestParam("postId") Long postId,
                           @RequestParam("postTitle") String postTitle,
                           @RequestParam("postCategoryId") Integer postCategoryId,
                           @RequestParam("postContent") String postContent,
                           @RequestParam("verifyCode") String verifyCode,
                           HttpSession httpSession) {
        BBSUser bbsUser = (BBSUser) httpSession.getAttribute(Constants.USER_SESSION_KEY);
        if (null == postId || postId < 0) {
            return ResultGenerator.genFailResult("postId 参数错误");
        }
        BBSPost temp = bbsPostService.getBBSPostById(postId);
        if (temp == null) {
            return ResultGenerator.genFailResult("postId 参数错误");
        }
        if (!bbsUser.getUserId().equals(temp.getPublishUserId())) {
            return ResultGenerator.genFailResult("非本人发帖，无权限修改");
        }
        if (!StringUtils.hasLength(postTitle)) {
            return ResultGenerator.genFailResult("postTitle 参数错误");
        }
        if (null == postCategoryId || postCategoryId < 0) {
            return ResultGenerator.genFailResult("postCategoryId 参数错误");
        }
        BBSPostCategory bbsPostCategory = bbsPostCategoryService.selectById(postCategoryId);
        if (null == bbsPostCategory) {
            return ResultGenerator.genFailResult("postCategoryId 参数错误");
        }
        if (!StringUtils.hasLength(postContent)) {
            return ResultGenerator.genFailResult("postContent 参数错误");
```

```
            }
            if (postTitle.trim().length() > 32) {
                return ResultGenerator.genFailResult("标题过长");
            }
            if (postContent.trim().length() > 100000) {
                return ResultGenerator.genFailResult("内容过长");
            }
            String kaptchaCode = httpSession.getAttribute(Constants.VERIFY_CODE_KEY) + "";
            if (!StringUtils.hasLength(kaptchaCode) || !verifyCode.equalsIgnoreCase(kaptchaCode))
{
                return
ResultGenerator.genFailResult(ServiceResultEnum.LOGIN_VERIFY_CODE_ERROR.getResult()
);
            }
            temp.setPostTitle(postTitle);
            temp.setPostContent(postContent);
            temp.setPostCategoryId(postCategoryId);
            temp.setPostCategoryName(bbsPostCategory.getCategoryName());
            temp.setLastUpdateTime(new Date());
            if (bbsPostService.updateBBSPost(temp) > 0) {
                httpSession.removeAttribute(Constants.VERIFY_CODE_KEY);//清空 session 中
的验证码信息
                return ResultGenerator.genSuccessResult();
            } else {
                return ResultGenerator.genFailResult("请求失败，请检查参数及账号是否有操作
权限");
            }
        }
}

@Controller
public class BBSUserController {

    @Resource
    private BBSUserService bbsUserService;

    @Resource
    private BBSPostService bbsPostService;
```

```
@Resource
private BBSPostCollectService bbsPostCollectService;

@Resource
private BBSPostCommentService bbsPostCommentService;

@GetMapping({"/login", "/login.html"})
public String loginPage() {
    return "user/login";
}

@GetMapping({"/register", "/register.html"})
public String registerPage() {
    return "user/reg";
}

@GetMapping("/userCenter/{userId}")
public String userCenterPage(HttpServletRequest request, @PathVariable("userId") Long
userId) {
    //基本用户信息
    BBSUser bbsUser = bbsUserService.getUserById(userId);
    if (bbsUser == null) {
        return "error/error_404";
    }
    //近期发布的帖子
    List<BBSPost> recentPostList = bbsPostService.getRecentPostListByUserId(userId);
    //近期回复的内容
    List<RecentCommentListEntity> recentCommentList =
bbsPostCommentService.getRecentCommentListByUserId(userId);
    request.setAttribute("bbsUser", bbsUser);
    request.setAttribute("recentPostList", recentPostList);
    request.setAttribute("recentCommentList", recentCommentList);
    return "user/home";
}

@GetMapping("/userSet")
public String userSetPage(HttpServletRequest request) {
```

```
        BBSUser currentUser = (BBSUser) request.getSession().getAttribute(Constants.USER_SESSION_KEY);
        request.setAttribute("bbsUser", currentUser);
        return "user/set";
    }

    @GetMapping("/myCenter")
    public String myCenterPage(HttpServletRequest request) {
        //基本用户信息
        BBSUser currentUser = (BBSUser) request.getSession().getAttribute(Constants.USER_SESSION_KEY);
        //我发的贴
        List<BBSPost> myBBSPostList = bbsPostService.getMyBBSPostList(currentUser.getUserId());
        int myBBSPostCount = 0;
        if (!CollectionUtils.isEmpty(myBBSPostList)) {
            myBBSPostCount = myBBSPostList.size();
        }
        //我收藏的贴
        List<BBSPost> collectRecords = bbsPostCollectService.getCollectRecordsByUserId(currentUser.getUserId());
        int myCollectBBSPostCount = 0;
        if (!CollectionUtils.isEmpty(collectRecords)) {
            myCollectBBSPostCount = collectRecords.size();
        }
        request.setAttribute("myBBSPostList", myBBSPostList);
        request.setAttribute("myBBSPostCount", myBBSPostCount);
        request.setAttribute("collectRecords", collectRecords);
        request.setAttribute("myCollectBBSPostCount", myCollectBBSPostCount);
        request.setAttribute("bbsUser", currentUser);
        return "user/index";
    }

    @PostMapping("/updateUserInfo")
    @ResponseBody
    public Result updateInfo(@RequestParam("nickName") String nickName,
                             @RequestParam("userGender") int userGender,
                             @RequestParam("location") String location,
```

```
                                @RequestParam("introduce") String introduce,
                                HttpSession httpSession) {

        if (!StringUtils.hasLength(nickName)) {
            return ResultGenerator.genFailResult("nickName 参数错误");
        }
        if (userGender < 1 || userGender > 2) {
            return ResultGenerator.genFailResult("userGender 参数错误");
        }
        if (!StringUtils.hasLength(location)) {
            return ResultGenerator.genFailResult("location 参数错误");
        }
        if (!StringUtils.hasLength(introduce)) {
            return ResultGenerator.genFailResult("introduce 参数错误");
        }
        BBSUser bbsUser = (BBSUser)
httpSession.getAttribute(Constants.USER_SESSION_KEY);
        bbsUser.setNickName(nickName);
        if (userGender == 1) {
            bbsUser.setGender("男");
        }
        if (userGender == 2) {
            bbsUser.setGender("女");
        }
        bbsUser.setLocation(location);
        bbsUser.setIntroduce(introduce);
        if (bbsUserService.updateUserInfo(bbsUser, httpSession)) {
            Result result = ResultGenerator.genSuccessResult();
            return result;
        } else {
            Result result = ResultGenerator.genFailResult("修改失败");
            return result;
        }
    }

    @PostMapping("/updateHeadImg")
    @ResponseBody
    public Result updateHeadImg(@RequestParam("userHeadImg") String userHeadImg,
```

```
                                    HttpSession httpSession) {

        if (!StringUtils.hasLength(userHeadImg)) {
            return ResultGenerator.genFailResult("userHeadImg 参数错误");
        }
        BBSUser bbsUser = (BBSUser) httpSession.getAttribute(Constants.USER_SESSION_KEY);
        bbsUser.setHeadImgUrl(userHeadImg);
        if (bbsUserService.updateUserHeadImg(bbsUser, httpSession)) {
            Result result = ResultGenerator.genSuccessResult();
            return result;
        } else {
            Result result = ResultGenerator.genFailResult("修改失败");
            return result;
        }
    }

    @PostMapping("/updatePassword")
    @ResponseBody
    public Result passwordUpdate(HttpServletRequest request, @RequestParam("originalPassword") String originalPassword,
                                 @RequestParam("newPassword") String newPassword) {
        if (!StringUtils.hasLength(originalPassword) || !StringUtils.hasLength(newPassword)) {
            return ResultGenerator.genFailResult("参数不能为空");
        }
        BBSUser currentUser = (BBSUser) request.getSession().getAttribute(Constants.USER_SESSION_KEY);
        if (bbsUserService.updatePassword(currentUser.getUserId(), originalPassword, newPassword)) {
            //修改成功后清空 session 中的数据，前端控制跳转至登录页
            request.getSession().removeAttribute(Constants.USER_SESSION_KEY);
            return ResultGenerator.genSuccessResult();
        } else {
            return ResultGenerator.genFailResult("修改失败");
        }
    }
```

```
    @GetMapping("/logout")
    public String logout(HttpSession httpSession) {
        httpSession.removeAttribute(Constants.USER_SESSION_KEY);
        return "user/login";
    }

    @PostMapping("/register")
    @ResponseBody
    public Result register(@RequestParam("loginName") String loginName,
                           @RequestParam("verifyCode") String verifyCode,
                           @RequestParam("nickName") String nickName,
                           @RequestParam("password") String password,
                           HttpSession httpSession) {
        if (!StringUtils.hasLength(loginName)) {
            return
ResultGenerator.genFailResult(ServiceResultEnum.LOGIN_NAME_NULL.getResult());
        }
        if (!PatternUtil.isEmail(loginName)) {
            return
ResultGenerator.genFailResult(ServiceResultEnum.LOGIN_NAME_NOT_EMAIL.getResult());
        }
        if (!StringUtils.hasLength(password)) {
            return
ResultGenerator.genFailResult(ServiceResultEnum.LOGIN_PASSWORD_NULL.getResult());
        }
        if (!StringUtils.hasLength(verifyCode)) {
            return
ResultGenerator.genFailResult(ServiceResultEnum.LOGIN_VERIFY_CODE_NULL.getResult())
;
        }
        String kaptchaCode = httpSession.getAttribute(Constants.VERIFY_CODE_KEY) + "";
        if (!StringUtils.hasLength(kaptchaCode) || !verifyCode.equalsIgnoreCase(kaptchaCode))
{
            return
ResultGenerator.genFailResult(ServiceResultEnum.LOGIN_VERIFY_CODE_ERROR.getResult()
);
        }
        String registerResult = bbsUserService.register(loginName, password, nickName);
```

```
        //注册成功
        if (ServiceResultEnum.SUCCESS.getResult().equals(registerResult)) {
            httpSession.removeAttribute(Constants.VERIFY_CODE_KEY);//删除 session 中
的验证码
            return ResultGenerator.genSuccessResult();
        }
        //注册失败
        return ResultGenerator.genFailResult(registerResult);
    }

    @PostMapping("/login")
    @ResponseBody
    public Result login(@RequestParam("loginName") String loginName,
                        @RequestParam("verifyCode") String verifyCode,
                        @RequestParam("password") String password,
                        HttpSession httpSession) {
        if (!StringUtils.hasLength(loginName)) {
            return
ResultGenerator.genFailResult(ServiceResultEnum.LOGIN_NAME_NULL.getResult());
        }
        if (!PatternUtil.isEmail(loginName)) {
            return
ResultGenerator.genFailResult(ServiceResultEnum.LOGIN_NAME_NOT_EMAIL.getResult());
        }
        if (!StringUtils.hasLength(password)) {
            return
ResultGenerator.genFailResult(ServiceResultEnum.LOGIN_PASSWORD_NULL.getResult());
        }
        if (!StringUtils.hasLength(verifyCode)) {
            return
ResultGenerator.genFailResult(ServiceResultEnum.LOGIN_VERIFY_CODE_NULL.getResult())
;
        }
        String kaptchaCode = httpSession.getAttribute(Constants.VERIFY_CODE_KEY) + "";
        if (!StringUtils.hasLength(kaptchaCode) || !verifyCode.equalsIgnoreCase(kaptchaCode))
{
            return
ResultGenerator.genFailResult(ServiceResultEnum.LOGIN_VERIFY_CODE_ERROR.getResult()
```

```
);
            }
            String loginResult = bbsUserService.login(loginName, MD5Util.MD5Encode(password,
"UTF-8"), httpSession);
            //登录成功
            if (ServiceResultEnum.SUCCESS.getResult().equals(loginResult)) {
                  httpSession.removeAttribute(Constants.VERIFY_CODE_KEY);//删除 session 中
的验证码
                  return ResultGenerator.genSuccessResult();
            }
            //登录失败
            return ResultGenerator.genFailResult(loginResult);
      }
}

@Controller
public class IndexController {

      @Resource
      private BBSPostCategoryService bbsPostCategoryService;

      @Resource
      private BBSPostService bbsPostService;

      @GetMapping({"", "/", "/index", "/index.html"})
      public String indexPage(HttpServletRequest request,
                                    @RequestParam(value = "categoryId", required = false) Long
categoryId,
                                    @RequestParam(value = "page", required = false, defaultValue
= "1") Integer page,
                                    @RequestParam(value = "keyword", required = false) String
keyword,
                                    @RequestParam(value = "period", required = false) String
period,
                                    @RequestParam(value = "orderBy", required = false) String
orderBy) {
            List<BBSPostCategory>                    bbsPostCategories                    =
bbsPostCategoryService.getBBSPostCategories();
```

```
        if (CollectionUtils.isEmpty(bbsPostCategories)) {
            return "error/error_404";
        }

        //第一步：将分类数据封装到 request 域中
        request.setAttribute("bbsPostCategories", bbsPostCategories);

        //第二步：将近期热议的帖子数据封装到 request 域中
        request.setAttribute("hotTopicBBSPostList",
bbsPostService.getHotTopicBBSPostList());

        //封装参数
        Map params = new HashMap();
        params.put("page", page);
        params.put("limit", 10);//默认每页 10 条
        //帖子类别
        if (categoryId != null && categoryId > 0) {
            request.setAttribute("categoryId", categoryId);
            params.put("categoryId", categoryId);
        }
        //搜索关键字
        if (StringUtils.hasLength(keyword)) {
            request.setAttribute("keyword", keyword);
            params.put("keyword", keyword);
        }
        //时间周期 (周榜、月榜、全部)
        if (StringUtils.hasLength(period)) {
            request.setAttribute("period", period);
            params.put("period", period);
        }
        //排序（按照阅读量或者时间排序）
        if (StringUtils.hasLength(orderBy)) {
            request.setAttribute("orderBy", orderBy);
            params.put("orderBy", orderBy);
        }
        PageQueryUtil pageUtil = new PageQueryUtil(params);

        //第三步：将帖子列表数据封装到 request 域中
```

```
        request.setAttribute("pageResult", bbsPostService.getBBSPostPageForIndex(pageUtil));
        return "index";
    }
}
```